

Maskinlæring for økonomer

Kompendium i SOK-3023

Markus J. Aase

Universitetslektor i matematikk og statistikk

November 2024
Handelshøgskolen UiT

"In the brain, you have connections between the neurons called synapses, and they can change. All your knowledge is stored in those synapses."
— Geoffrey Hinton

Om kompendiet

Dette kompendiet handler om maskinlæring, og er designet for økonomistudenter på masternivå. Kurset forutsetter grunnleggende matematikk, statistikk, og programmeringsferdigheter i Python. Dog, kreves det ikke at studentene har mye kjennskap til maskinlæring og AI fra før. Det er kunnskap jeg håper studentene vil tilegne seg gjennom dette kurset, hvor dette kompendiet, sammen med de fysiske forelesningene vil gi en god innføring til maskinlæring - i teori og praksis.

Læring kan være en krevende prosess, men den er også full av muligheter og belønninger. Å forstå maskinlæring vil ikke bare styrke dine analytiske ferdigheter, men også gi deg et fortrinn i dagens og fremtidens arbeidsmarked, der datadrevne beslutninger blir stadig viktigere.

La oss begynne reisen inn i maskinlæringens fascinerende verden, lykke til!
— Markus J. Aase

Innhold

1	Introduksjon til maskinl�ring	6
2	Grunnleggende matematikk og statistikk	7
2.1	N�dvendig line�r algebra for maskinl�ring	7
2.1.1	Operasjoner med vektorer og matriser	7
2.1.2	Tensorer og matriser	9
2.2	N�dvendig kalkulus for maskinl�ring	11
2.3	N�dvendig statistikk for maskinl�ring	13
2.3.1	Forventningsverdi og varians	13
2.3.2	Reducible og irreducible error	14
2.3.3	Aktiveringsfunksjoner	15
3	Noen maskinl�ringsprinsipper	17
3.1	Veiledet og ikke-veiledet l�ring	17
3.1.1	Veiledet l�ring (supervised learning)	17
3.1.2	Ikke-veiledet l�ring (Unsupervised learning)	17
3.1.3	Reinforcement learning	17
3.2	Klassifikasjon versus regresjon	18
3.2.1	Klassifikasjon	18
3.2.2	Regresjon	18
3.2.3	Forskjeller mellom klassifikasjon og regresjon	18
3.3	Prediksjon versus inferens	19
3.4	Bias-variance tradeoff	19
3.5	Overfitting og underfitting	20
3.6	Evalueringsmetoder	20
3.6.1	Confusion matrix	20
3.6.2	Accuracy	21
3.6.3	Precision og Recall	21
3.6.4	AUC og ROC-kurver	22
3.6.5	R-Squared	22
3.7	Tapsfunksjon/Kostfunksjon	23
3.7.1	Gradient descent	24
3.8	Trenings, validerings og test-sett	25
4	Noen "enkle" maskinl�ringsteknikker	26
4.1	Line�r regresjon	26
4.2	Logistisk regresjon	28
4.2.1	Teoretisk grunnlag	28
4.3	Beslutningstr�r	28
4.3.1	Splitting av feature Space	28
4.3.2	Pruning av beslutningstr�r	30
4.3.3	Random forests	31
4.3.4	Boosting	31

5	Dypl�ring	33
5.1	Nevrale nettverk	33
5.1.1	Struktur av et nevralt nettverk	33
5.2	Nevrale nettverk	34
5.2.1	Arkitektur og matematikken	34
5.2.2	Feed-forward, "fully-connected" nevrale nettverk	36
5.2.3	Verkt�y og biblioteker	37
5.2.4	Bruksomr�der i �konomi	37
5.3	Whisper-modellen	38
5.3.1	Transformer-modell	38
5.3.2	Personvern	39
5.3.3	Fra lyd til innsikt: Sentimentanalyse	39
5.3.4	Praktisk bruk: Modellst�rrelser og Hardware	39
5.4	Bildeklassifisering	40
5.4.1	Bilder og deres datastruktur	41
5.4.2	Convolutional Neural Networks	41
5.5	Tidsseriemodeller	44
5.5.1	Recurrent neural networks	44
5.5.2	LSTM-modeller	44
5.5.3	N�r brukes dette?	45
6	S� hvorfor er dette interessant for �konomer	46
7	Hvorfor er maskinl�ring interessant for �konomer?	46
7.1	Prediksjon som et beslutningsverkt�y	46
7.2	Den ustrukturerte datarevolusjonen	46
7.3	Kompleksitet og ikke-linearitet	47
8	Praktisk bruk i Python	49
8.1	Datasett	49
8.2	Innlasting av data	49
8.3	Utforske dataen	50
8.4	Dataprosessering	50
8.5	Splitte data i trenings- og testsett	52
8.6	Line�r regresjon	53
8.7	Logistisk regresjon	54
8.8	Beslutningstr�r	55
8.9	Random Forests	56
8.10	Boosting	56
8.11	Nevrale nettverk	56
8.11.1	Epochs	56
8.11.2	Regulariseringstekikker	56
8.12	Whisper	56
8.13	CNN	56
8.14	LSTM	56

9	Oppgaver	57
9.1	Oppgaver	57
9.2	Løsningsforslag	59

1 Introduksjon til maskinlæring

Maskinlæring er et omfattende felt som i stor grad handler om å lage modeller som kan lære fra data, og gjøre prediksjoner eller ta beslutninger basert på dem. I hovedsak kan maskinlæring deles inn i ulike teknikker som har til felles at de forsøker å estimere en underliggende funksjon fra data. Dette inkluderer metoder som er spesifikke for ulike typer data, som språkmodeller for tekst eller bildeklassifisering for visuelle data. I dette kurset vil vi utforske flere av disse teknikkene, med fokus på både teori og praktiske anvendelser.

Vi starter med en innføring i hva maskinlæring (ML) og kunstig intelligens (AI) er, og ser på anvendelser innenfor finans og økonomi. Kurset vil ta for seg hva som skiller prediksjon fra inferens, og gir en forståelse av sentrale konsepter som *reducible* og *irreducible* feil, samt forskjellen mellom regresjon og klassifikasjon. Vi vil også introdusere de matematiske fundamentene som trengs for å forstå ML, som lineær algebra, tensor-operasjoner, sannsynlighetsteori og derivasjon.

Når vi går dypere inn i maskinlæring, kommer vi til å fokusere på nevrale nettverk (NN). Dette omfatter alt fra nettverksarkitektur, som hvordan ulike lag er organisert, til funksjoner som styrer læringsprosessen, slik som tapsfunksjon (loss function) og aktiviseringsfunksjoner. Gjennom eksempler som MNIST-datasettet for håndskrevne tall, vil vi se hvordan nevrale nettverk kan trenes for å gjøre prediksjoner. Vi vil også dykke inn i avanserte teknikker, som språkmodeller og bildeklassifisering. Et eksempel på en språkmodell er Whisper, som kan brukes til å transkribere tale til tekst. Slike modeller er basert på dyp læring og viser kraften til nevrale nettverk i naturlig språkbehandling. Når det gjelder bildeklassifisering, vil vi utforske hvordan nevrale nettverk kan trenes til å gjenkjenne objekter i bilder, og vi vil bruke evalueringsmetrikker som confusion matrix, og dens evalueringsmetrikker, for å si noe om hvor god eller dårlig modellen vår er.

Videre ser vi på anvendelser innen tidsserieanalyse, spesielt med fokus på LSTM-modeller (Long Short-Term Memory). Disse modellene er spesielt nyttige når vi skal analysere sekvensielle data, som finansielle tidsserier, der sammenhengen mellom data på ulike tidspunkter er viktig. LSTM-modeller kan håndtere lange avhengigheter i data og blir brukt mer og mer i økonomiske prognoser.

Målet med dette kompendiet er å være et teori-supplement til forelesningene og Jupyter notatene vi går igjennom i Google Colab, for å knytte teori opp mot praksis. Kurset krever derfor aktiv deltakelse i forelesningene, og egenstudium av kompendiet. *Maskinlæring for økonomer*, har som mål å gi masterstudenter i samfunnsøkonomi et *et innblikk* i maskinlæringsverden, hva det kan brukes til og potensielle anvendelser innen samfunnsøkonomien.

2 Grunnleggende matematikk og statistikk

For   kunne gj re noe som helst innenfor maskinl ring, trenger man noen grunnleggende matematikk og statistikk-kunnskaper.

2.1 N dvendig line r algebra for maskinl ring

Line r algebra er grunnleggende for mange maskinl ringsalgoritmer. Her er noen n kkelkonsepter:

- **Vektorer:** En vektor er en ordnet liste av tall som kan representere datapunkter i et flerdimensjonalt rom. En vektor $\mathbf{x}$ i n -dimensjoner kan skrives som:

$$\mathbf{x} = \begin{pmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{pmatrix}$$

hvor x_i er komponentene i vektoren. Vektorer brukes til   representere egenskaper ved datapunkter og utf re beregninger.

- **Matriser:** En matrise er et rektangul rt array av tall og kan brukes til   representere datasett og transformasjoner. En matrise $\mathbf{A}$ av dimensjoner $m \times n$ kan skrives som:

$$\mathbf{A} = \begin{pmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m1} & a_{m2} & \cdots & a_{mn} \end{pmatrix}$$

Matriseoperasjoner som addisjon, subtraksjon og multiplikasjon er viktige for   kombinere og transformere data.

- **Normer:** Normen til en vektor $\mathbf{x}$ er en m ling av dens lengde. To vanlige normer er L2-norm (euklidisk norm) og L1-norm:

$$L2\text{-norm: } \|\mathbf{x}\|_2 = \sqrt{\sum_{i=1}^n x_i^2}$$

$$L1\text{-norm: } \|\mathbf{x}\|_1 = \sum_{i=1}^n |x_i|$$

Normer brukes i optimalisering for   evaluere og minimere tap. Dette vil vi komme mer tilbake til senere i kompendiumet.

2.1.1 Operasjoner med vektorer og matriser

I maskinl ring og datavitenskap er vektorer og matriser grunnleggende byggesteiner. De brukes til   representere data, modeller og beregninger. Her vil vi se n rmere p  noen viktige operasjoner som involverer vektorer og matriser.

Transponering Transponering av en matrise er en operasjon som bytter plass p  radene og kolonnene. Hvis vi har en matrise A definert som:

$$A = \begin{pmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \end{pmatrix}$$

da er den transponerte matrisen A^T :

$$A^T = \begin{pmatrix} a_{11} & a_{21} \\ a_{12} & a_{22} \\ a_{13} & a_{23} \end{pmatrix}$$

Transponering er nyttig i mange sammenhenger, blant annet i beregningene av dot-produktet mellom vektorer og i tilpassingen av data for videre analyser.

Skalar multiplikasjon Skalar multiplikasjon er en operasjon der en matrise eller vektor multipliseres med et tall (skalar). Hvis vi har en vektor x definert som:

$$x = \begin{pmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{pmatrix}$$

og en skalar k , s  er den resulterende vektoren fra skalar multiplikasjon $k \cdot x$:

$$k \cdot x = \begin{pmatrix} k \cdot x_1 \\ k \cdot x_2 \\ \vdots \\ k \cdot x_n \end{pmatrix}$$

Skalar multiplikasjon brukes ofte til   justere vektene i maskinl ringsmodeller. For eksempel kan det brukes til   justere l ringshastigheten i algoritmer som gradient descent, hvor vi oppdaterer vektene i forhold til den gradienten av tapsfunksjonen.

Matrisemultiplikasjon Matrisemultiplikasjon er en annen viktig operasjon i maskinl ring. Gitt to matriser A og B , der A er en $m \times n$ matrise og B er en $n \times p$ matrise, kan produktet $C = AB$ v re definert som:

$$C_{ij} = \sum_{k=1}^n a_{ik} b_{kj}$$

Resultatet C vil da v re en $m \times p$ matrise. Matrisemultiplikasjon er ofte brukt til   kombinere funksjoner og data i nevrale nettverk. For eksempel kan vi bruke matriser til   representere vektene i et nevralt nettverk, hvor inngangsdataene multipliseres med vektmatrisen for   produsere aktiveringsverdier i det skjulte laget.

Eksempel: La oss se p  et konkret eksempel med to matriser:

$$A = \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}, \quad B = \begin{pmatrix} 5 & 6 \\ 7 & 8 \end{pmatrix}$$

Her er A en 2×2 matrise og B er ogs  en 2×2 matrise. For   finne produktet $C = AB$, bruker vi formelen for matrisemultiplikasjon:

$$C_{11} = (1 \cdot 5) + (2 \cdot 7) = 5 + 14 = 19$$

$$C_{12} = (1 \cdot 6) + (2 \cdot 8) = 6 + 16 = 22$$

$$C_{21} = (3 \cdot 5) + (4 \cdot 7) = 15 + 28 = 43$$

$$C_{22} = (3 \cdot 6) + (4 \cdot 8) = 18 + 32 = 50$$

Dermed f r vi produktmatrisen C :

$$C = AB = \begin{pmatrix} 19 & 22 \\ 43 & 50 \end{pmatrix}$$

I nevrale nettverk kan vi tenke oss at matrise A representerer inputverdiene (for eksempel egenskaper ved et datasett), og matrise B representerer vektene som er l rt gjennom treningsprosessen. Produktet C gir oss de aktiverte verdiene som deretter kan brukes i neste lag av nettverket. Dette vil vi komme mer tilbake til i detalj senere i kompendiet.

Hvorfor lagre data som vektorer og matriser? Data lagres ofte som vektorer og matriser fordi disse formatene g r det lettere   utf re matematiske operasjoner og transformasjoner. Vektorer kan representere funksjoner, egenskaper eller observasjoner av datapunkter, mens matriser kan brukes til   representere grupper av slike datapunkter. Denne strukturen tillater effektiv utnyttelse av line r algebra, noe som er grunnleggende for algoritmer innen maskinl ring og dyp l ring. Videre er mange biblioteker og rammeverk, inkludert TensorFlow, optimalisert for beregninger med matriser og tensorer, noe som g r databehandlingen raskere og mer effektiv. Lagres data i et vektor- eller matriseformat, muligg r det parallellbehandling, noe som er essensielt for   h ndtere store datamengder og komplekse modeller i maskinl ring [Hull, 2021, s. 20-21].

2.1.2 Tensorer og matriser

En **tensor** er en generell betegnelse p  en datastruktur som brukes mye innen maskinl ring og numerisk databehandling. Vi kan se p  en tensor som en utvidelse av skalarer, vektorer og matriser til h yere dimensjoner. Tensorer gir en fleksibel m te   representere data med flere dimensjoner, som er nyttig i nevrale nettverk og bildebehandling. Vi baserer oss her p  en definisjon fra maskinl ringslitteraturen [Goodfellow et al., 2016]:

“In the general case, an array of numbers arranged on a regular grid with a variable number of axes is known as a tensor.”

Typer av tensorer En tensor kan betraktes som en generalisering av matriser og vektorer til h yere dimensjoner. Dette er en grunnleggende datastruktur i maskinl ring, spesielt i rammeverk som PyTorch og TensorFlow. Tensorer deles ofte inn etter rang (rank), som beskriver antallet dimensjoner:

- **Skalar** (rank-0 tensor): En enkeltverdi, som et tall 3 eller 7.5. Representert som $\mathbb{R}$.

- **Vektor** (rank-1 tensor): En rekke med verdier, for eksempel en liste som $[1, 3, 5, 7]^T$. Representert som $\mathbb{R}^n$.
- **Matrise** (rank-2 tensor): En tabell med verdier, for eksempel en $\mathbb{R}^3 \times \mathbb{R}^3$ matrise:

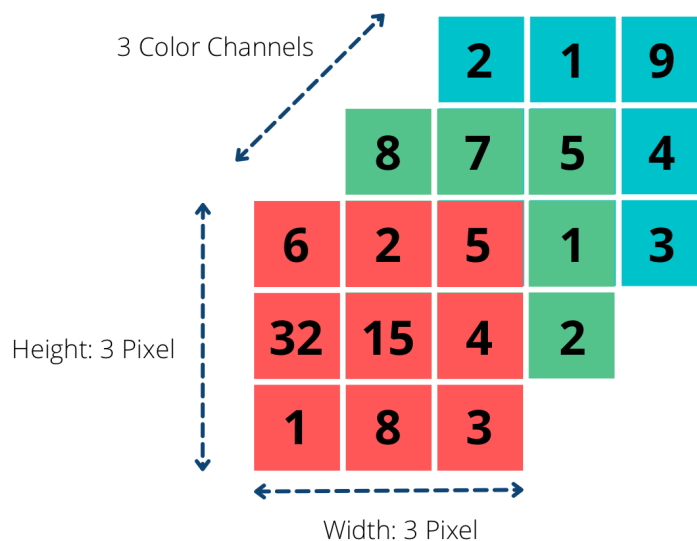
$$\begin{bmatrix} 3 & 5 & 7 \\ 2 & 6 & 1 \\ 8 & 4 & 9 \end{bmatrix}$$

Representert som $\mathbb{R}^n \times \mathbb{R}^m$.

- **Rank-3 tensor** og h yere: En flerdimensjonal generalisering av matriser. For eksempel kan en 3D-tensor tenkes som en “kubelignende” struktur med flere matriser stablet opp  hverandre. Representert som $\mathbb{R}^n \times \mathbb{R}^m \times \mathbb{R}^p$.

N r brukes h yere rang (rank) tensorer?

- Et gr skalabilde (med  n kanal) representeres vanligvis som en rank-2 tensor, alts  en 2D-matrise av pikselverdier der dimensjonene er h yde $\times$ bredde.
- Et f rgebilde, som har tre kanaler (r d, gr nn og bl ), representeres som en rank-3 tensor der dimensjonene er h yde $\times$ bredde $\times$ kanaler. Se Figur 1



Figur 1: Illustrerer en rank-3 tensor, som ofte er et f rgebilde best ende av r d, gr nn og bl  [Lang, 2022].

Det viktigste   forstå er at n r vi snakker om en *tensor* i maskinl ring, refererer **rangen** til antall dimensjoner, og dette m  ikke forveksles med dimensjonene til selve matrisen!

2.2 N dvendig kalkulus for maskinl ring

Kalkulus er avgj rende for   forst  hvordan maskinl ringsmodeller blir trent og optimalisert. Viktige konsepter inkluderer:

- **Derivasjon:** Derivasjon er en metode for   finne hastigheten til endring av en funksjon. For en funksjon $f(x)$ er den deriverte definert som:

$$f'(x) = \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h}$$

Derivasjon brukes i gradient descent-algoritmen for   finne minimum av en tapsfunksjon [Hull, 2021, s. 39].

- **Gradienten:** Gradienter er vektorer av partielle deriverte som viser retningen for den bratteste stigningen av en funksjon. For en funksjon $f(\mathbf{x})$ med flere variabler, hvor $\mathbf{x} = (x_1, x_2, \dots, x_n)^T$, er gradienten:

$$\nabla f(\mathbf{x}) = \begin{pmatrix} \frac{\partial f}{\partial x_1} \\ \frac{\partial f}{\partial x_2} \\ \vdots \\ \frac{\partial f}{\partial x_n} \end{pmatrix}$$

Gradient descent-algoritmen oppdaterer modellparametre i retningen av den negative gradienten for   minimere tapsfunksjonen:

$$\mathbf{x}_{i+1} = \mathbf{x}_i - \eta \cdot \nabla f(\mathbf{x}_i)$$

hvor η er l ringsraten, og x_i er modellparameter, og x_{i+1} er den oppdaterte modellparameteren.

Eksempel p  beregning av gradienten: La oss si vi har en funksjon:

$$f(x_1, x_2) = x_1^2 + 3x_2^2$$

For   finne gradienten, beregner vi de partielle derivatene:

$$\frac{\partial f}{\partial x_1} = 2x_1$$

$$\frac{\partial f}{\partial x_2} = 6x_2$$

Dermed er gradienten:

$$\nabla f(\mathbf{x}) = \begin{pmatrix} 2x_1 \\ 6x_2 \end{pmatrix}$$

Oppdatering med gradient descent: Anta at vi starter med initialverdiene $x_1 = 1$ og $x_2 = 2$ og bruker en l ringsrate p  $\eta = 0.1$. Vi kan beregne gradienten:

$$\nabla f(1, 2) = \begin{pmatrix} 2 \cdot 1 \\ 6 \cdot 2 \end{pmatrix} = \begin{pmatrix} 2 \\ 12 \end{pmatrix}$$

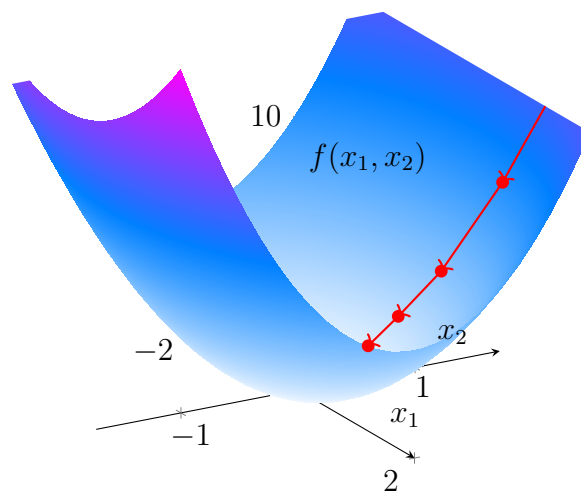
N  oppdaterer vi parameterne:

$$\begin{pmatrix} x_1 \\ x_2 \end{pmatrix}_{i+1} = \begin{pmatrix} x_1 \\ x_2 \end{pmatrix}_i - 0.1 \cdot \begin{pmatrix} 2 \\ 12 \end{pmatrix}$$

$$\begin{pmatrix} x_1 \\ x_2 \end{pmatrix}_{i+1} = \begin{pmatrix} 1 \\ 2 \end{pmatrix} - \begin{pmatrix} 0.2 \\ 1.2 \end{pmatrix} = \begin{pmatrix} 0.8 \\ 0.8 \end{pmatrix}$$

S  etter  n oppdatering har vi f tt $x_1 = 0.8$ og $x_2 = 0.8$. Denne prosessen kan gjentas flere ganger til vi konvergerer mot minimum av funksjonen. Denne funksjonen kan v re hva vi velger den til   v re, ofte bruker vi det vi kaller for en *tapsfunksjon* eller *kostfunksjon*, men det vil vi se n rmere p  etter hvert.

Likevel, ta en ekstra titt p  det vi akkurat har gjort her, for hvis du forst r dette - er vi p  god vei mot   forst  noe av det mest fundamentale i maskinl ring og nevrale nettverk.



Figur 2: Illustrerer gradient decent p  $f(x_1, x_2) = x_1^2 + 3x_2^2$ med initielle verdier $(x_1, x_2) = (1, 2)$ og vi ser at punktene g r mot minimumspunktet $(0, 0)$.

2.3 N dvendig statistikk for maskinl ring

Statistikk gir verkt yene som trengs for   analysere data og trekke konklusjoner. N kkelkonsepter inkluderer:

- **Sannsynlighet:** Sannsynlighet er et m l p  usikkerhet og kan representeres som et tall mellom 0 og 1. For en hendelse A , er sannsynligheten $P(A)$ definert som:

$$P(A) = \frac{\text{Antall gunstige utfall}}{\text{Totalt antall utfall}}$$

Sannsynlighetsfordelinger, som normalfordelingen, brukes til   modellere data og gj re prediksjoner.

- Sannsynlighet er essensielt i maskinl ring fordi mange modeller handler om   forutsi usikkerhet og h ndtere st y i data. For eksempel n r vi skal forutsi en ukjent verdi Y gitt noen observasjoner X , kan vi for eksempel bruke logistisk regresjon, som bygger direkte p  sannsynlighetsmodeller for   gi prediksjoner. Videre er begreper som forventningsverdi, varians og fordelingsfunksjoner sentrale for   modellere hvordan data oppf rer seg og for   evaluere modellens ytelse.

2.3.1 Forventningsverdi og varians

- **Forventningsverdi:** Forventningsverdien (eller gjennomsnittet) av en stokastisk variabel X er et m l p  den sentrale tendensen av variabelen. Den er definert som:

$$E[X] = \sum_{i=1}^n x_i P(X = x_i) \quad (\text{diskret variabel})$$

eller

$$E[X] = \int_{-\infty}^{\infty} x f(x) dx \quad (\text{kontinuerlig variabel})$$

Forventningsverdien gir et m l p  hva som kan v re den forventede verdien fra et datasett.

- **Varians:** Variansen $\text{Var}(X)$ m ler hvor mye data sprer seg rundt forventningsverdien. Den er definert som:

$$\text{Var}(X) = E[(X - E[X])^2]$$

Variansen kan ogs  uttrykkes som:

$$\text{Var}(X) = E[X^2] - (E[X])^2$$

En lav varians indikerer at dataene er n r forventningsverdien, mens h y varians indikerer st rre spredning. Varians er kritisk for   forst  over- og underfitting i modeller.

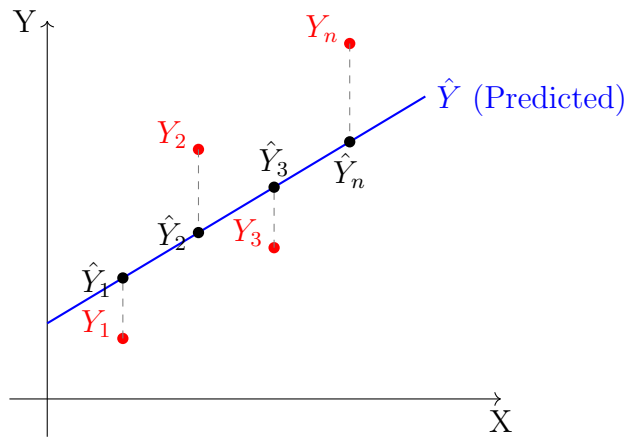
2.3.2 Reducible og irreducible error

Reducible og irreducible error er to fundamentale konsepter i maskinl ring som beskriver de ulike kildene til feil i en modell. For   forst  disse konseptene bedre, bruker vi matematikk til   forklare dem.

La oss f rst definere **Mean Squared Error (MSE)**, som er en ofte brukt metrikk for   m le ytelsen til en modell. Hvis en vektor av n prediksjoner er generert fra et utvalg med n datapunkter, der Y_i er den observerte verdien og $\hat{Y}_i$ er den predikerte verdien, kan MSE uttrykkes som:

$$MSE = \frac{1}{n} \sum_{i=1}^n (Y_i - \hat{Y}_i)^2$$

MSE representerer gjennomsnittet av kvadrerte forskjeller mellom observerte verdier og predikerte verdier, se Figur 3. Det som er viktig   merke seg her, er at feilene i modellen kan stamme fra to hovedkilder: **reducible error** og **irreducible error** [James et al., 2023, s. 17-18].



Figur 3: Illustrerer Y_i og $\hat{Y}_i$, $i = 1, 2, \dots, n$ og differansen mellom observasjonene og prediksjonene.

Prediksjon er ofte et m l i maskinl ring. N r vi pr ver   modellere forholdet mellom den avhengige variabelen Y (den observerte responsen) og uavhengige variabler X (funksjoner), kan vi uttrykke det slik:

$$Y = f(X) + \epsilon$$

hvor $f(X)$ er den "sanne" funksjonen som beskriver forholdet mellom X og Y , og ϵ er feilleddet eller st yen som ikke kan forklares av X . Prediksjonen vi gjør fra modellen v r kan uttrykkes som $\hat{Y} = \hat{f}(X)$, hvor $\hat{f}(X)$ er v r tiln rming til den sanne funksjonen $f(X)$.

Et viktig poeng er at det alltid er en viss mengde st y ϵ i systemet, ogs  kalt **irreducible error**, som vi ikke kan modellere. Dette feilleddet har egenskapene:

$$E[\epsilon] = 0$$

og

$$\text{Var}(\epsilon) = \sigma^2$$

Dette betyr at gjennomsnittet av st yen er null, men variansen σ^2 er positiv, og representerer uforutsigbar variasjon i dataene.

For   forst  hvordan den totale feilen kan splittes i reducible og irreducible komponenter, kan vi se p  forventningen til kvadratet av feilen mellom den faktiske verdien Y og den predikerte verdien $\hat{Y}$:

$$E[(Y - \hat{Y})^2] = E[(f(X) + \epsilon - \hat{f}(X))^2]$$

Uttrykket over kan deles opp som:

$$E[(Y - \hat{Y})^2] = E[(f(X) - \hat{f}(X))^2] + E[\epsilon^2]$$

Siden ϵ er st yen, vet vi at $E[\epsilon^2] = \text{Var}(\epsilon)$ (Se teori om varians over, for hvorfor det blir slik). Dermed f r vi:

$$E[(Y - \hat{Y})^2] = \underbrace{(f(X) - \hat{f}(X))^2}_{\text{Reducible Error}} + \underbrace{\text{Var}(\epsilon)}_{\text{Irreducible Error}}$$

Her ser vi at den totale feilen deles inn i to deler:

$(f(X) - \hat{f}(X))^2$ representerer **reducible error**, som er forskjellen mellom den sanne funksjonen $f(X)$ og v r prediksj n $\hat{f}(X)$. Denne feilen kan reduseres ved   forbedre modellen, for eksempel ved   velge bedre funksjoner, justere hyperparametere eller bruke en bedre egnet modell.

$\text{Var}(\epsilon)$ representerer **irreducible error**, som er variansen til st yen ϵ . Denne feilen kan ikke reduseres, da den kommer fra naturlig variasjon i dataene som ikke kan forklares av modellen. Dette er et ekstremt viktig konsept   ha forst else for n r vi jobber med data og maskinl ring.

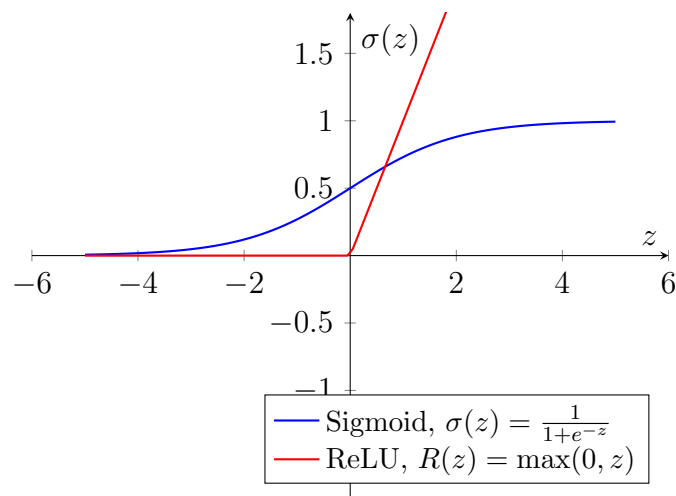
2.3.3 Aktiveringsfunksjoner

I nevrale nettverk brukes aktiveringsfunksjoner for   introdusere ikke-linearitet i modellene, slik at de kan l re komplekse m nstre i dataene. Uten en aktiveringsfunksjon vil hvert lag i et nevralt nettverk v re en line r kombinasjon av forrige lag, noe som begrenser modellens evne til   l re ikke-line re relasjoner.

Det finnes flere typer aktiveringsfunksjoner, og hver har ulike egenskaper som kan v re fordelaktige avhengig av oppgaven. Noen av de mest brukte aktiveringsfunksjonene er:

- **Sigmoid-funksjonen**: En S-formet kurve som begrenser utgangsverdiene til   v re i spennet $[0, 1]$. Den brukes ofte i utgangslag n r vi vil ha en sannsynlighetsverdi mellom 0 og 1, som passer bra for bin r klassifisering.
- **ReLU (Rectified Linear Unit)**: En funksjon som lar positive inputverdier passere uendret, men setter negative verdier til null. ReLU er popul r i dype nevrale nettverk fordi den g r det enkelt og raskt   l re, men kan noen ganger f re til "d de" nevroner som slutter   reagere hvis de f r for mange nullverdier.

Det er mange aktiveringsfunksjoner man kan velge, men Figur 4 illustrerer grafene til to av de vanligste aktiveringsfunksjonene.



Figur 4: Vanlige aktiveringsfunksjoner: Sigmoid og ReLU.

3 Noen maskinl ringsprinsipper

3.1 Veiledet og ikke-veiledet l ring

Maskinl ring kan i hovedsak deles inn i to hovedkategorier: veiledet l ring og ikke-veiledet l ring. I tillegg, har vi noe som kalles *reinforcement learning*, men det vil vi ikke g  s rlig inn p  her.

3.1.1 Veiledet l ring (supervised learning)

Veiledet l ring inneb rer at modellen trenes med en datasett som inneholder b de inngangsdata (features) og tilh rende korrekte utdata (labels) [James et al., 2023, s. 25]. M let er at modellen l rer seg   kartlegge en funksjon som kan brukes til   predikere utdataene basert p  nye, usette inngangsdata. Et typisk eksempel p  veiledet l ring er klassifikasjonsproblemer, som n r man fors ker   klassifisere blomsterarter basert p  fysiske m linger (f.eks. lengde og bredde p  kronblad).

En vanlig veiledet l ringsmodell er *line r regresjon*, som fors ker   finne den beste line re sammenhengen mellom input $X = (x_1, x_2, \dots, x_n)$ og output y . Modellen kan da uttrykkes som:

$$y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_n x_n + \epsilon$$

hvor ϵ er st yen eller feilen i modellen, og β_i er de ukjente koeffisientene modellen fors ker   estimere.

3.1.2 Ikke-veiledet l ring (Unsupervised learning)

I ikke-veiledet l ring har man kun tilgang til inngangsdataene, og det finnes ingen korrekte utdata eller labels   l re fra. M let er her   oppdage m nstre eller strukturer i dataene. Et vanlig eksempel er klyngeanalyse (clustering), hvor modellen fors ker   gruppere datapunktene i klynger basert p  likhet.

En popul r metode innen ikke-veiledet l ring er *K-Means Clustering*, hvor m let er   dele et datasett inn i k klynger basert p  avstanden mellom datapunkter. K-means algoritmen fors ker   minimere summen av kvadrerte avstander mellom hvert datapunkt og sentrum av sin respektive klynge.

3.1.3 Reinforcement learning

Forsterkende l ring er en annen viktig teknikk innen maskinl ring, hvor en *agent* l rer   ta beslutninger gjennom interaksjon med et milj . I motsetning til veiledet og ikke-veiledet l ring, der modellen l rer fra statiske datasett, f r agenten tilbakemelding i form av bel nninger eller straff basert p  handlingene den tar. M let er   maksimere den totale bel nningen over tid ved   l re seg en optimal handlingsstrategi.

Selv om forsterkende l ring er en kraftfull teknikk, er dette ikke fokus for dette kompendiet, som prim rt tar for seg **veiledet l ring**.

3.2 Klassifikasjon versus regresjon

Klassifikasjon og regresjon er to grunnleggende metoder innen maskinl ring, og de har forskjellige m l og metoder for   behandle data.

3.2.1 Klassifikasjon

Klassifikasjon refererer til prosessen med   tilordne kategoriske outputdata til inputdata. M let med klassifikasjon er   forutsi hvilken klasse (eller kategori) et gitt datapunkt tilh rer. Dette kan v re bin r klassifikasjon, hvor det er to klasser, eller flerkategoriklassifikasjon, hvor det er flere klasser.

Eksempler p  klassifikasjonsoppgaver inkluderer:

- E-post filtrering (spam eller ikke-spam)
- Diagnose av sykdommer basert p  symptomer
- Bildeklassifisering (f.eks. klassifisere bilder som hunder eller katter)

Klassifikasjonsmodeller lærer fra historiske data for   lage en prediksjonsmodell, ofte ved   bruke teknikker som beslutningstr r, nevrale nettverk, logistisk regresjon eller st ttevektor-maskiner (SVM). Det handler nemlig om *prediktoren* er *kvantitativ* eller *kvalitativ*.

3.2.2 Regresjon

Regresjon, derimot, er en oppgave som handler om   forutsi kontinuerlige verdier. M let med regresjon er   modellere forholdet mellom en uavhengig variabel (eller flere variabler) og en avhengig variabel (ogs  kalt prediktor). Dette kan v re en enkel regresjon, der man forutsier en enkelt utfall basert p  en enkelt input, eller multippel regresjon, der flere inputvariabler brukes til   forutsi en enkelt utfall.

Eksempler p  regresjonsoppgaver inkluderer:

- Forutsi huspriser basert p  st rrelse, beliggenhet og andre faktorer
- Forutsi aksjekurser over tid
- Estimere inntekten basert p  utdanningsniv  og erfaring

Regresjonsmodeller bruker metoder som line r regresjon, polynomregresjon eller mer komplekse modeller som nevrale nettverk for   tilpasse seg dataene.

3.2.3 Forskjeller mellom klassifikasjon og regresjon

Den prim re forskjellen mellom klassifikasjon og regresjon ligger i outputvariabelen som predikeres:

- **Klassifikasjon:** Outputvariabelen er **kategoriske** (f.eks. klasse A, B eller C).
- **Regresjon:** Outputvariabelen er **kontinuerlige verdier** (f.eks. 10.5, 200.75).

Valget mellom klassifikasjon og regresjon avhenger av problemstillingen og typen data som er tilgjengelig. For   oppsummere, klassifikasjon brukes for   tilordne datapunkter til ulike klasser (kategoriske), mens regresjon brukes for   forutsi numeriske verdier basert p  inputdata.

3.3 Prediksjon versus inferens

I maskinl ring er det viktig   skille mellom prediksjon og inferens.

- **Prediksjon** refererer til prosessen med   forutsi fremtidige utfall basert p  tilgjengelige data, alts  historiske data vi har samlet inn og gitt *labels*, alts  *merket* utfallet. For eksempel, i en regresjonsmodell kan vi bruke tidligere data til   forutsi verdien av en avhengig variabel Y gitt en uavhengig variabel X :

$$Y = f(X) + \epsilon$$

hvor $f(X)$ er den prediktive funksjonen, og ϵ representerer st yen i dataene.

- **Inferens** fokuserer derimot p    trekke konklusjoner om de underliggende prosessene som genererer dataene. Det inneb rer   evaluere forholdet mellom variabler og kan inkludere estimering av parameterne i modellen. For eksempel, i en line r regresjonsmodell kan vi bruke metoder som minste kvadraters estimat for   bestemme parameterne:

$$\hat{\beta} = (X^T X)^{-1} X^T y$$

hvor $\hat{\beta}$ er parameterestimatene, X er designmatrisen, og y er vektoren av observerte utfall.

Her, vil vi fokusere mest p  prediksjon.

3.4 Bias-variance tradeoff

Bias-variance tradeoff er et sentralt konsept i maskinl ring som beskriver balansen mellom to typer feil som p virker modellens ytelse:

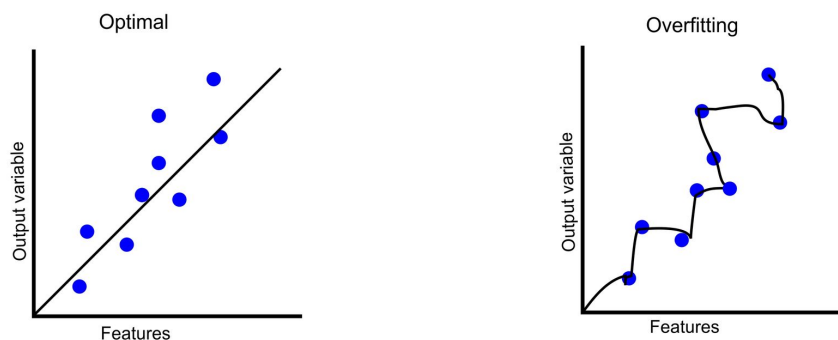
- **Bias** refererer til feil som oppst r n r en modell forenkler virkeligheten for mye. Dette kan f re til at modellen ikke fanger opp de underliggende m nstrene i dataene, noe som resulterer i *underfitting*. En modell med h y bias kan for eksempel v re en line r modell (som line r regresjon) som fors ker   tilpasse seg data som har en ikke-line r struktur.
- **Variance** refererer til feil som oppst r n r modellen er for kompleks og fanger opp st y i dataene. Dette kan f re til *overfitting*, der modellen tilpasser seg treningsdataene for godt, men har d rlig generaliseringsevne p  nye data. En modell med h y varians kan for eksempel v re en kompleks nevralt nettverk med mange lag, eller et enkelt beslutningstre som har vokst veldig dypt (mange grener).

M let er   finne en balanse mellom bias og varians som minimerer den totale feilen:

$$\text{Total Error} = \text{Bias}^2 + \text{Variance} + \text{Irreducible Error}$$

3.5 Overfitting og underfitting

Overfitting oppst r n r en modell er for kompleks og tilpasser seg treningsdataene for mye, noe som resulterer i d rlig ytelse p  testdata. Dette kan skje n r modellen har for mange parametere eller n r det er for lite treningsdata tilgjengelig, se Figur 5. For   identifisere overfitting kan vi se p  forskjellen mellom trenings- og testfeil; en modell som presterer mye bedre p  treningsdataene enn p  testdataene er sannsynligvis overfitted.



Figur 5: Illustrerer hva en god tilpasning er, og hva overfitting er [Ogbemi, 2021].

Underfitting er det motsatte av overfitting, der modellen er for enkel til   fange opp m nstrene i dataene. Underfitting kan oppst  n r man bruker en for enkel modell, som en line r regresjon p  ikke-line re data, eller n r det er utilstrekkelig med trening. Resultatet blir d rlig ytelse b de p  trenings- og testdata.

For   unng  b de overfitting og underfitting, kan man bruke teknikker som:

- Regularisering (f.eks. Lasso, Ridge)
- Kryssvalidering
- Dropout-teknikker
- Tidlig stopp under trening av nevrale nettverk
- ... og mange, mange fler

3.6 Evaluering av modeller

Evaluering av modeller er avgj rende for   vurdere hvor godt en modell fungerer. Det finnes flere metoder for   evaluere ytelsen til en maskinl ringsmodell.

3.6.1 Confusion matrix

Confusion matrix er et verkt y som brukes for   oppsummere hvor god eller d rlig modellen v r er til   klassifisere noe. For et bin rt klassifiseringsproblem, vil en confusion matrix vise antall sanne positive (TP), sanne negative (TN), falske positive (FP) og falske negative (FN). En typisk 2x2 confusion matrix ser slik ut:

	Predicted Negative	Predicted Positive
Actual Negative	TN	FP
Actual Positive	FN	TP

Fra denne confusion matrixen kan vi utlede flere evalueringsmetrikker, som for eksempel:

- **Accuracy:**

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

- **Precision:**

$$\text{Precision} = \frac{TP}{TP + FP}$$

- **Recall** (ogs  kjent som Sensitivitet):

$$\text{Recall} = \frac{TP}{TP + FN}$$

Det er mange m ter   evaluere en maskinl ringsmodell sin ytelse, og her diskuterer vi bare et f tall av noen av de mest populære metrikkene. Vit at det finnes mange fler, og hva som egner seg best - jo, det kommer an p  dataene dine og problemet du pr ver   l se.

3.6.2 Accuracy

Accuracy er en av de mest brukte metrikker for   evaluere ytelsen til klassifiseringsmodeller. Den defineres som andelen korrekt klassifiserte tilfeller (b de sanne positive og sanne negative) i forhold til det totale antallet tilfeller. Den kan beregnes som f lger:

$$\text{Accuracy} = \frac{\text{Antall korrekte prediksjoner}}{\text{Totalt antall prediksjoner}} = \frac{TP + TN}{TP + TN + FP + FN}$$

Selv om accuracy er enkel   forst , kan den v re misvisende i tilfeller der datasettene er ubalanserte. I slike tilfeller kan det v re bedre   bruke andre metrikker, som precision og recall.

3.6.3 Precision og Recall

Precision er et m l p  hvor mange av de predikerte positive tilfellene som faktisk er positive. Det er spesielt nyttig i situasjoner der kostnaden for falske positive er h y. Den kan beregnes med formelen:

$$\text{Precision} = \frac{TP}{TP + FP}$$

Recall (eller Sensitivitet) er et m l p  hvor mange av de faktiske positive tilfellene som ble korrekt identifisert. Det er nyttig i tilfeller der kostnaden for falske negative er h y. Den kan beregnes som:

$$\text{Recall} = \frac{TP}{TP + FN}$$

I praksis er det ofte en avveining mellom precision og recall, kjent som F1-score, som gir en balansert vurdering av begge:

$$F1 = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

3.6.4 AUC og ROC-kurver

En sentral del av maskinlæring er å kunne evaluere hvor godt en modell presterer, og det finnes flere evalueringsmetoder for dette formålet. Blant de mest brukte evalueringsmetodene for klassifikasjonsmodeller finner vi ROC (Receiver Operating Characteristic) og AUC (Area Under Curve).

- **ROC-kurve:** ROC-kurven er et verktøy for å visualisere ytelsen til en klassifikasjonsmodell, spesielt når målet er å skille mellom to klasser (for eksempel “positiv” og “negativ”). Kurven plotter forholdet mellom *sant positiv rate* (True Positive Rate, TPR) og *falsk positiv rate* (False Positive Rate, FPR) for ulike terskelverdier. TPR representerer andelen korrekte positive prediksjoner blant de faktiske positive, mens FPR representerer andelen feilaktige positive prediksjoner blant de faktiske negative. En ROC-kurve som bøyer seg nært opp mot øvre venstre hjørne (TPR=1, FPR=0) viser en modell med høy prediksjonsevne.
- **AUC-kurve** AUC, eller *Area Under the Curve*, er et numerisk mål for å evaluere modellen basert på ROC-kurven. AUC representerer arealet under ROC-kurven og varierer mellom 0 og 1, der 1 indikerer en perfekt modell, og 0.5 indikerer en modell som ikke presterer bedre enn ren gjetning. En høy AUC-verdi betyr at modellen har god separasjonsevne, altså at den kan skille mellom de to klassene på en pålitelig måte. Generelt sett anses en AUC på 0.7–0.8 som akseptabel, 0.8–0.9 som god, og over 0.9 som utmerket.

ROC-kurven og AUC gir en verdifull innsikt i hvordan modellen presterer på ulike terskelverdier. Ved å analysere ROC-kurven kan man forstå balansen mellom TPR og FPR, og AUC-verdi gir et intuitivt mål på totalytelsen. Disse evalueringsmetodene er spesielt nyttige i situasjoner med skjeve datasett (f.eks. hvor det er mange flere negative enn positive eksempler), da de gir mer innsikt enn enkle nøyaktighetsmål alene.

3.6.5 R-Squared

R^2 , eller **R-squared** (ofte kalt **forklaringsgraden** på norsk), er en statistisk målestørrelse som brukes til å vurdere hvor godt en regresjonsmodell forklarer variasjonen i et datasett. R^2 -verdien representerer andelen av den totale variasjonen i den avhengige variabelen som kan forklares av modellen.

Formel for R^2 R^2 -verdien beregnes vanligvis som følger:

$$R^2 = 1 - \frac{SS_{\text{res}}}{SS_{\text{tot}}}$$

hvor:

- SS_{res} er summen av kvadrerte residualer, dvs. forskjellen mellom de observerte verdiene og de verdiene som er predikert av modellen.
- SS_{tot} er summen av kvadrerte avvik mellom de observerte verdiene og gjennomsnittet av de observerte verdiene.

Det kan altså skrives på følgende måte:

$$R^2 = 1 - \frac{\sum_i (y_i - \hat{f}_i)^2}{\sum_i (y_i - \bar{y})^2}$$

Tolking av R^2

- **$R^2 = 1$:** Modellen forklarer all variasjon i dataene perfekt.
- **$R^2 = 0$:** Modellen forklarer ingen av variasjonene i dataene (prediksjonene er like gjennomsnittsverdien til den avhengige variabelen).
- **R^2 mellom 0 og 1:** Modellen forklarer en viss andel av variasjonen i dataene, der høyere verdier indikerer bedre tilpasning.

Begrensninger ved R^2

- **Ikke alltid et mål på "goodness of fit" i modellen:** En høy R^2 -verdi betyr ikke nødvendigvis at modellen er god eller generaliserbar; det kan bety at modellen er overtilpasset.
- **Vanskelig i ikke-lineære modeller:** R^2 kan være misvisende for komplekse, ikke-lineære modeller der variasjonen ikke forklares særlig bra av enkle regresjonsmodeller.
- **Kan øke med flere variabler:** Å legge til flere forklaringsvariabler i modellen øker ofte R^2 -verdien, selv om disse variablene kanskje ikke er relevante.

Justert R^2 For å håndtere noen av begrensningene ved R^2 brukes ofte en justert R^2 -verdi. Denne tar hensyn til antall forklaringsvariabler i forhold til mengden data og straffer for overtilpasning, noe som gir en mer pålitelig evaluering av modellens forklaringskraft når mange variabler er involvert.

R^2 er nyttig for å vurdere en regresjonsmodells forklaringsgrad, men bør brukes sammen med andre evalueringsmetoder for en fullstendig vurdering av modellens ytelse.

3.7 Tapsfunksjon/Kostfunksjon

I maskinlæring er tapsfunksjonen (eller kostfunksjonen) et mål på hvor godt modellen presterer i forhold til de faktiske utfallene. Den kvantifiserer forskjellen mellom de predikerte verdiene ($\hat{Y}$) fra modellen og de faktiske verdiene (Y) i datasettet. En vanlig brukt tapsfunksjon for regresjonsproblemer er den *gjennomsnittlige kvadratiske feilen* (eng: Mean Squared Error, MSE), som defineres som:

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (Y_i - \hat{Y}_i)^2$$

Her er n antall datapunkter, Y_i er den faktiske verdien, og $\hat{Y}_i$ er den predikerte verdien. Ved   minimere tapsfunksjonen, pr ver vi   forbedre modellens n yaktighet.

3.7.1 Gradient descent

Gradient descent er en optimaliseringsalgoritme som brukes for   minimere tapsfunksjoner. Algoritmen fungerer ved   iterativt justere modellparametere (som vektorer og biaser i nevrale nettverk, som vi kommer mer inn p  senere) i retning av den negative gradienten av tapsfunksjonen. Dette gj r det mulig   finne det "optimale" punktet der tapsfunksjonen er minimal.

Gradienten av en kostfunksjon C er en vektor som inneholder de partielle deriverte av C med hensyn til hver av parametrene, som vektor $\mathbf{w}$. Matematisk kan gradienten skrives som:

$$\nabla C(\mathbf{w}) = \left(\frac{\partial C}{\partial w_1}, \frac{\partial C}{\partial w_2}, \dots, \frac{\partial C}{\partial w_k} \right)$$

hvor $\mathbf{w} = (w_1, w_2, \dots, w_k)$ er vektorene med parametere (ofte vektorer, eller weights). Oppdateringen av parametrene i gradient descent skjer if lge f lgende formel:

$$\mathbf{w}_{i+1} \leftarrow \mathbf{w}_i - \eta \cdot \nabla C(\mathbf{w}_i)$$

Her er η l ringsraten, som bestemmer hvor store skritt vi tar i retning av gradienten.   velge en passende l ringsrate er viktig, da en for h y rate kan f re til *overshooting*, mens en for lav rate kan f re til treningsprosessen tar lang tid.

  minimere tapsfunksjonen er essensen i maskinl ring, spesielt i nevrale nettverk der vi har mange vektorer og biaser. Ved   bruke gradient descent til   oppdatere disse parametrene kan vi l re komplekse m nstre i dataene og dermed fange opp sammenhenger vi mennesker aldri kunne funnet!

Stochastic gradient decent Stokastisk Gradient Descent (SGD) er en optimaliseringsmetode som ofte brukes i maskinl ring for   trene modeller, spesielt nevrale nettverk. I motsetning til tradisjonell gradient descent, som oppdaterer vektene basert p  hele treningsdatasettet, oppdaterer SGD vektene ved   bruke bare ett enkelt eksempel (eller en liten batch) om gangen - og denne velges ut tilfeldig.

Prosessen kan beskrives i f lgende trinn:

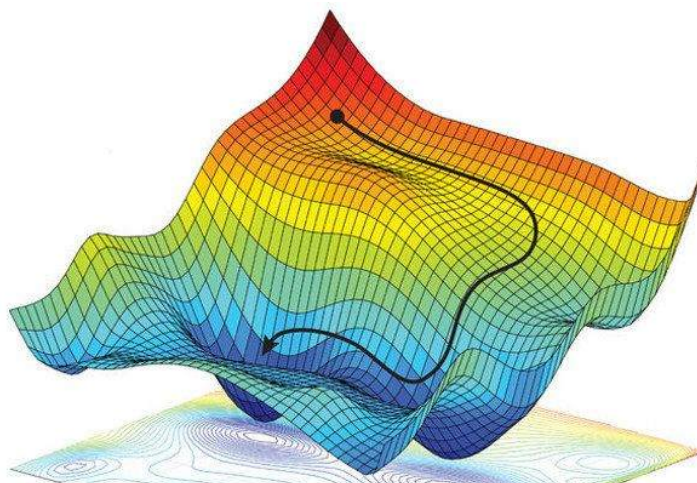
1. For hvert treningspunkt $(\mathbf{x}, y)$, beregnes gradienten av tapsfunksjonen L med hensyn til vektene $\mathbf{w}$:

$$\nabla L(\mathbf{w}) = \frac{\partial L}{\partial \mathbf{w}}$$

2. Vektene oppdateres ved hjelp av gradienten:

$$\mathbf{w} = \mathbf{w} - \eta \cdot \nabla L(\mathbf{w})$$

hvor η er l ringsraten, som bestemmer st rrelsen p  oppdateringene.



Figur 6: Illustrerer stokastisk gradient decent, hvor vi oppdaterer vektene v re for   konvergere mot et minimum [Mishra, 2023].

3. Prosessen gjentas for alle treningspunkter, noe som gjør at modellen kan l re gradvis og tilpasse seg dataene.

Dermed skiller stokastisk gradient decent (SDG) seg fra *vanlig* gradient decent. Fordi vanlig gradient decent, s  oppdaterer modellen vektene sine basert p  gradienten regnet ut av HELE treningssettet - dette krever mye datakraft. Mens SDG oppdaterer vektene sine ved   bruke kun  n (eller en liten *batch*) av treningssettet av gangen. Dette betyr at hver oppdatering av vektene er basert p  en tilfeldig valgt datapunkt fra datasettet. Dette introduserer variasjon i oppdateringen, og kan forhindre   ende opp i *lokale minimum*. Oftest f rer dette til raskere konvergering mot minimum, men kan ogs  ha noen implikasjoner. Men dette er et eget fagstoff i seg selv, og et sted m  vi stoppe.

Fordeler Stokastisk Gradient Descent har flere fordeler:

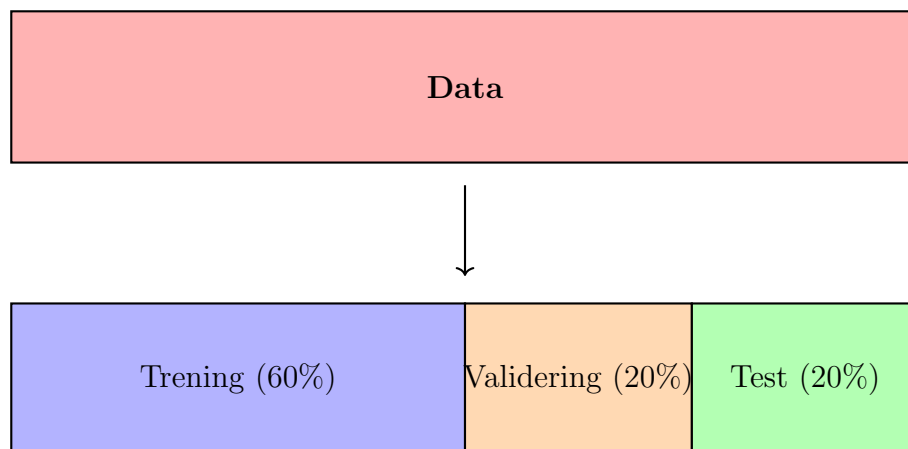
- **Effektivitet:** Den krever mindre minne og kan h ndtere store datasett raskere enn tradisjonell gradient descent.
- **Regularisering:** Den stokastiske biten av oppdateringene kan bidra til   unng  overfitting ved   introdusere "st y" i l ringsprosessen.

3.8 Trenings, validerings og test-sett

N r vi arbeider med maskinl ring, er det viktig   dele dataene v re opp i forskjellige sett for   evaluere modellens ytelse p  en p litelig m te. De vanligste m tene   splitte dataene p  er trening og testdata. Treningdata brukes til   trene modellen, mens testdata brukes til   evaluere hvor godt modellen generaliserer til nye, usette data.

Noen ganger er det ogs  fordelaktig   inkludere et valideringssett i tillegg til trening og testdata, se Figur 7. Valideringssettet brukes til   tune hyperparametrene i modellen og gir en indikasjon p  hvordan modellen vil prestere p  testdataene. Ved   bruke et valideringssett, kan vi redusere risikoen for *overtilpasning* (overfitting), som kan oppst  n r modellen tilpasser seg treningdataene for godt og dermed mister evnen til   generalisere.

Treningsprosessen, er den mest krevende prosessen i maskinlæring, og krever da oftest mest data. I Figur 7 er det satt til 60%, mens validering og test sett er satt til 20% - dette er dog ingen fasit, og opp til brukeren å bestemme disse andelene.



Figur 7: Illustrasjon av hvordan vi kan dele opp data inn i trening, validering og test sett.

4 Noen ”enkle” maskinlæringsteknikker

I kurset, vil vi fokusere mest på dyplæringsteknikker. Likevel, er det nødvendig å ha en forståelse av enklere maskinlæringsteknikker. Dette er teknikker som ofte kan være svært robuste, da det mest kompliserte ikke nødvendigvis er det beste!

De teknikkene vi kommer til å nevne nå, er mulige teknikker dere kan ta i bruk i kursets prosjektoppgave.

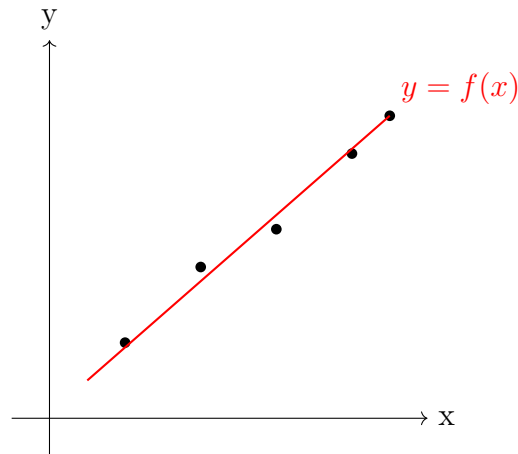
4.1 Lineær regresjon

Lineær regresjon er en av de mest grunnleggende og enkle teknikkene innen maskinlæring. Den brukes til å modellere forholdet mellom en uavhengig variabel x og en avhengig variabel y ved å tilpasse en rett linje til dataene. Dette egner seg når den avhengige variabelen er en numerisk verdi, og ikke kategori (da bør man bruke en klassifiseringsmetode). Den matematiske formelen for lineær regresjon er:

$$y = \beta_0 + \beta_1 x + \epsilon$$

hvor:

- y er den avhengige variabelen.
- x er den uavhengige variabelen.
- β_0 er konstantleddet (intercept).
- β_1 er stigningstallet (koeffisienten).
- ϵ er feilleddet.



Figur 8: Illustrasjon av line r regresjon. De svarte punktene representerer datapunktene, mens den r de linjen viser den beste tilpassede regresjonslinjen $y = f(x)$.

Figur 8 illustrerer hvordan enkel line r regresjon (eng: *Simple Linear Regression*) fungerer. Den viser en rekke datapunkter, og finner den beste line re sammenhengen mellom avhengig variabel og den uavhengige variabelen. N r vi har flere uavhengige variabler snakker vi om *multippel line r regresjon*.

Line r regresjon er en enkel, men kraftig teknikk for prediksjon og analyse. Den er mye brukt i mange felt, fra  konomi til ingeni rvitenskap, og danner grunnlaget for mer komplekse maskinl ringsmetoder. Likevel, hender det ofte at "det enkle er ofte det beste" - og pr ve en line r regresjon f r andre l ringsteknikker er en fin trening i forst else av datasettet man skal pr ve   gj re prediksjoner med.

4.2 Logistisk regresjon

Logistisk regresjon er en statistisk metode som brukes for   modellere sannsynligheten for at en bestemt klasse eller hendelse inntreffer, spesielt i tilfeller der utfallsvariabelen er kategorisk (f.eks. ja/nei, 0/1). Den er en type veiledet l ring som brukes i klassifisering-soppgaver.

4.2.1 Teoretisk grunnlag

Logistisk regresjon bygger p  prinsippet om   bruke en logistisk funksjon (ogs  kjent som sigmoid-funksjon) for   transformere line re kombinasjoner av inngangsvariabler til en sannsynlighet. Den logistiske funksjonen er definert som:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

hvor z er en line r kombinasjon av inngangsvariablene, gitt ved:

$$z = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_n x_n$$

Her representerer β_0 interceptet (konstantleddet) og $\beta_1, \beta_2, \dots, \beta_n$ er koeffisientene for de respektive funksjonene $x_1, x_2, \dots, x_n$.

Den predikerte sannsynligheten for at den avhengige variabelen Y er lik 1, gitt X , kan uttrykkes som:

$$P(Y = 1|X) = \sigma(z)$$

For   forutsi klassen til et datapunkt, bruker vi en terskelverdi (ofte 0.5). Hvis $P(Y = 1|X) \geq 0.5$, klassifiseres datapunktet som tilh rende klasse 1; ellers klassifiseres det som tilh rende klasse 0.

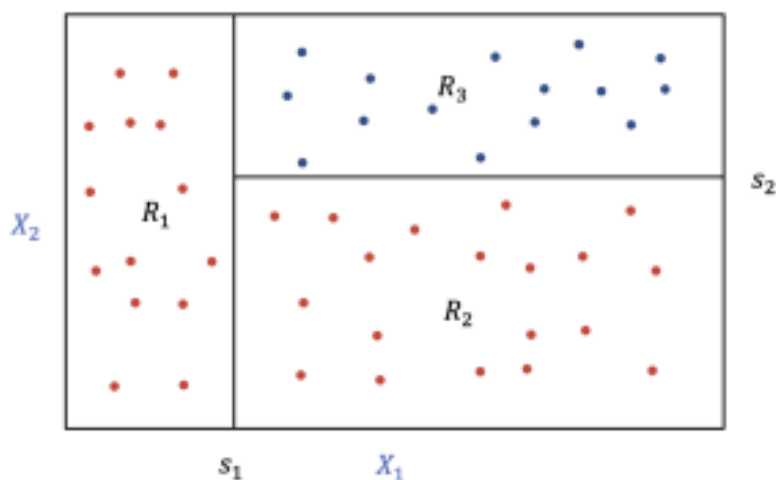
4.3 Beslutningstr r

Beslutningstr r er en popul r metode innen veiledet l ring for b de klassifikasjon og regresjon. De fungerer ved   dele opp data i grupper basert p  en rekke regler som ligner p  menneskelige avgj relser. Beslutningstr r gir en intuitiv og lettfattelig modell som kan visualiseres, noe som gj r det enklere   forst  hvordan modellen tar avgj relser. Beslutningstr r baserer seg p  f  antakelser sammenlignet med andre maskinl ringsteknikker, og produserer ofte gode resultat. Spesielt n r vi kombinerer de med andre statistiske konsepter, som bootstrapping [James et al., 2023].

4.3.1 Splitting av feature Space

Beslutningstr r baserer seg p  en teknikk hvor man splitter opp *feature space* i ikke-overlappende regioner. Hvis vi tar en  n-dimensjonal vektor med to prediktorer $\mathbf{X} = (X_1, X_2)$, og vi deler opp feature-space inn i $R_1, R_2, \dots, R_m$ distinkte, ikke-overlappende regioner, se Figur 9. For hver observasjon X_i som faller innenfor en spesifikk region R_j gj r vi *samme* prediksj n for den X_i verdien.

Det er alt for mange mulige oppdeling av feature space, til at vi kan vurdere alle ulike versjoner. Det ville rett og slett v rt for *beregningskrevende*.  n l sning er det som p 



Figur 9: Illustrasjon av feature space.[Aase, 2022].

engelsk kalles *recursive binary splitting*. Dette betyr at den f rste *splitten* i treet lages ved punkt s_1 , se Figur 9. Dette lager en *root node*, hvor responsen er enten st rre eller lik til verdien ved splittelse, alts  s_1 , eller leave. Da har vi delt opp feature space inn i $R_1 = [X|X_1 < s_1]$ og $R_2 = [X|X_1 \geq s_1]$. Denne s_1 er valgt i henhold til et *splitte-kriterie*, som igjen kommer an p  om vi har regresjon eller klassifikasjonsproblem foran oss.

Denne prosessen fortsetter rekursivt ved   splitte partisjonene R_j inn i flere og flere regioner, $j = 1, 2, \dots$. Hver R_j representerer en *terminal node*, som gir en prediksj n. I Figur 9 er R_1 og R_2 de r de punktene, mens R_3 er de bl  punktene.

Eksempel

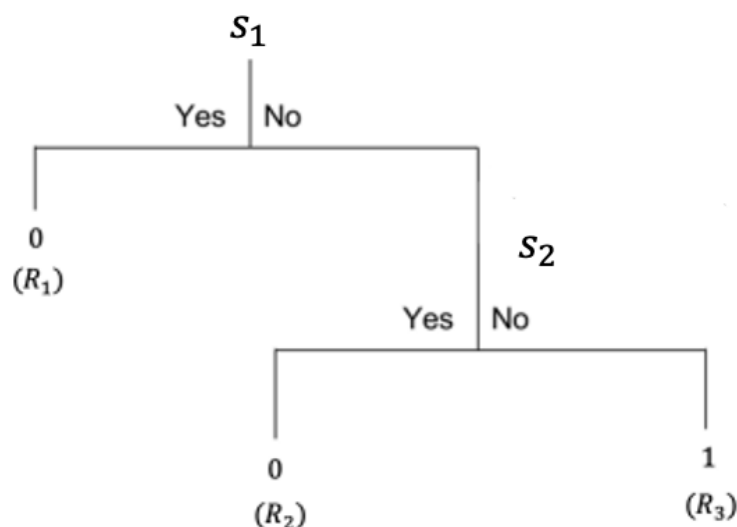
Hvis vi ser p  figuren over, kan de r de punktene representere "ikke-svindel", og de bl  punktene representerer "svindel". Da kan X_1 v re hvor mye penger som er i en transaksjon i Euro. S  si at $s_1 = 4000$  , da betyr det alt alt til venstre for den vertikale linjen ved s_1 betyr "ikke-svindel", som vi kan si er 0. Ogs  kan X_2 v re antall transaksjoner siste m ned, og hvis $s_2 = 3$, vil det betyr at regionen R_3 er omr det hvor transaksjonen har v rt **over** 4000 , og denne personen har gjort **flere** enn 3 transaksjoner denne m neden, og dette tilsvarer verdien 1.

Dette kan vi visualisere enklere med et beslutningstre, se Figur 10.

Viktig

Her gir vi en kort oversikt av hva beslutningstr er er for noe. Det finnes mer teori bak disse l ringsteknikkene, og ting er ulikt for regresjon- og klassifiseringsproblemer. Noen stikkord, for den interesserte er:

- Gini index, cross-entropy, deviance
- Cost complexity pruning
- Cross validation



Figur 10: Illustrasjon av et beslutningstre.[Aase, 2022].

- RSS

4.3.2 Pruning av beslutningstr r

Pruning (beskj ring) er en teknikk som brukes for   redusere st rrelsen p  et beslutningstre ved   fjerne noder som ikke bidrar betydelig til modellens ytelse. Dette er viktig fordi:

1. **Reduksjon av Overfitting:** Store tr r kan overtilpasse treningsdataene, noe som resulterer i d rlig generalisering til nye, usette testdata.
2. **Forbedret Modellkompleksitet:** Enkle modeller er ofte lettere   tolke og kan forbedre beslutningstaking i praksis. Et tre med hundrevis av beslutninger er vanskeligere   tolke, enn for eksempel 10.

Pruning kan ses p  som en form for regularisering, da det bidrar til   kontrollere kompleksiteten i modellen. Regulariseringsteknikker er avgj rende i maskinl ring for   unng  overtilpasning.

Dette kan vi heldigvis l se p  mange m ter i Python, som for eksempel

- **Maksimal dybde (max_depth):** Setter en grense for hvor dypt treet kan vokse. Dette er en av de enkleste regulariseringsmetodene for beslutningstr r. Et grunt tre kan for rsake underfitting, mens et veldig dypt tre kan f re til overfitting, s  det er avgj rende   finne en balansert dybde av treet sitt - og dette er jobben til den som utvikler modellen!
- **Minimum antall samples for splitting (min_samples_split):** Denne hyperparameteren sikrer at en node m  ha et minimum antall samples f r den kan splittes. Dette kan forhindre sm  splittelser som fanger st y og dermed f rer til en d rligere modell.
- **Maksimalt antall features (max_features):** Ved   sette en grense for antallet

features/variabler som vurderes for splitting, kan du legge til en form for feature-regularisering.

- Det er enda flere. Se dokumentasjon i Python for alle alternativer.

4.3.3 Random forests

I forrige kapittel skr t vi av beslutningstr r og dens intuitive, og enkle tolkning. Som dere kanskje har skj nt til n , handler mye i maskinl ring og valg av teknikker om en *trade-off*. Det er med andre ord, vanskelig   f  *pose og sekk*.

Beslutningstr r har en  penbar svakhet som er at de ofte kan f re til en h y *varians*. Et forklarende eksempel, er at hvis vi bruker to ulike treningssett, fra samme data, kan de to beslutningstr rne v re veldig ulik en annen. Denne variansen kan vi redusere ved   kombinere beslutningstr r med en metode vi kaller *bootstrapping*, der vi bygger flere modeller p  ulike, tilfeldige utvalg av treningsdata.

Random Forests er en *ensemble-metode* som bygger p  dette prinsippet. Her skaper vi en samling av beslutningstr r (en *skog*) ved hjelp av bootstrapping, og avgj relsen tas basert p  flertallsstemmen fra alle tr rne i skogen. Dette f rer til lavere varians og  kt robusthet, noe som gj r Random Forests til et effektivt verkt y for b de klassifiserings- og regresjonsoppgaver.

Algorithm 1 Random forest algoritme for klassifiseringstr r

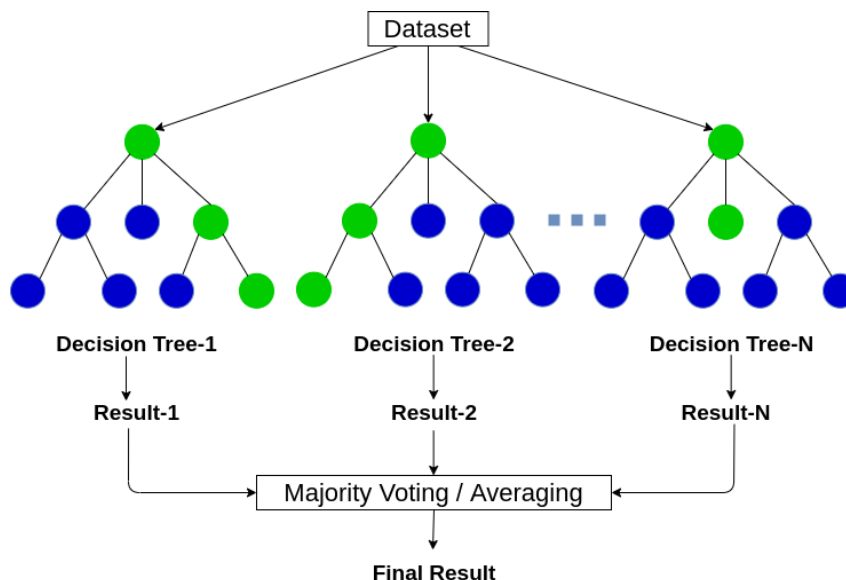
```
0: Splitt datasettet  $\mathbf{X}$  til trening  $\mathbf{X}^{\text{train}}$  og test sett  $\mathbf{X}^{\text{test}}$ .
0: for  $B$  ganger (antall bootstrap) do
0:   Bootstrap  $\mathbf{X}^{\text{train}} \Rightarrow \mathbf{X}_1^{\text{train}}, \dots, \mathbf{X}_B^{\text{train}}$ 
0: end for
0: for hvert bootstrapped datasett  $\mathbf{X}_b^{\text{train}}$  do
0:   Lag tr r basert p   $\mathbf{X}_1^{\text{train}}, \dots, \mathbf{X}_B^{\text{train}} \Rightarrow \hat{f}_1(x), \dots, \hat{f}_B(x) \{m = \sqrt{p}\}$ 
0: end for
0: Sett sammen beslutningstr rne til  $T_B^{\text{RF}} = [\hat{f}_1(x), \dots, \hat{f}_B(x)]$ 
0: Prediksjonen/output er majority vote, alts  det flest tr r predikerer. =0
```

I algoritmen over, er $m = \sqrt{p}$, hvor m er antall prediktorer som vurderes i hver split i treet og p er totalt antall prediktorer. Dette er et vanlig valg, for   bidra til reduksjon av variansen. Hadde vi satt $m = p$ har vi en teknikk som kalles *bagging* [James et al., 2023].

NB: Husk at vi har diskutert **klassifisering** over her, dog selv om regresjon behandles relativt likt, er det ulikheter. Dette dekker vi ikke i kompendiet her, men er nyttig   vite om til prosjektoppgave, masteroppgave eller hvis man kommer over problemstillinger senere - hvor man  nsker   benytte seg av beslutningstr r og *regresjon*.

4.3.4 Boosting

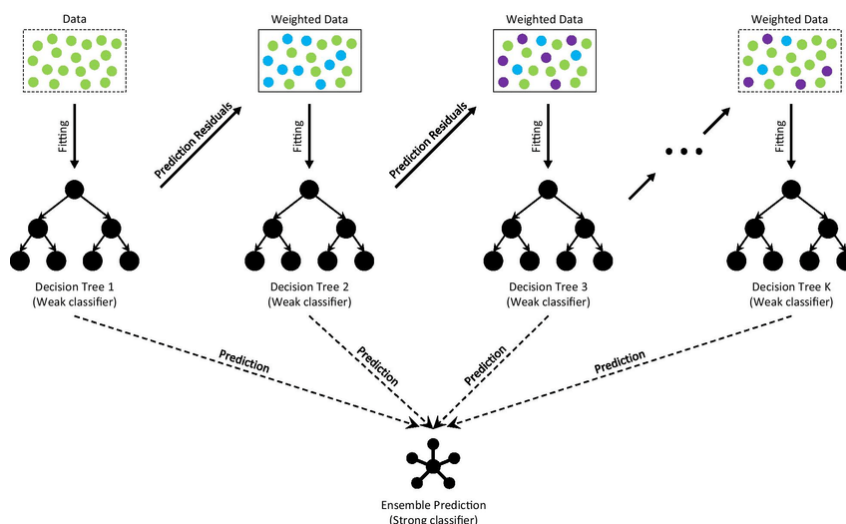
Boosting er en statistisk l ringsalgoritme som kan anvendes p  ulike metoder. N r boosting anvendes p  tr r, benyttes mange beslutningstr r (klassifikasjon eller regresjon) som kombineres til et mer robust og bedre prediktivt verkt y. P  denne m ten har boosting et lignende form l som tidligere diskuterte metoder (random forests). Til tross for denne likheten, skiller boosting seg betydelig fra bagging og random forests [Hastie et al., 2009].



Figur 11: Illustrasjon av random forests.[Curry, 2021].

I de to sistnevnte metodene vokser tr ene uavhengig av hverandre p  separate bootstrap-treningsdata (se Figur 11), mens boosting f lger en annen tiln rming der tr ene bygges sekvensielt, se Figur 12. Med dette menes at hvert tre bruker informasjon fra det forrige treet [James et al., 2023]. Dette er illustrert i Figur 12, der vi har et initielt tre, og dette oppdateres til at mange "weak classifiers" blir til en "strong classifier".

Vi g r ikke inn i detaljer her, men dette er en sv rt god maskinl ringsteknikk som ofte produserer gode resultater. Ved   bruke boosting, m  man ogs  gj re en del *tuning*, som betyr at man har flere parametere man m  justere for   hindre overtilpasning og undertilpasning. I Python, er det veldokumentert hvordan man kan tilpasse ulike parametere som *learning rate*, *antall tr er* og *interaction depth*.



Figur 12: Illustrasjon av boosting av beslutningstr er.[Deng et al., 2021].

5 Dyplæring

Dyplæring er et omfattende fagfelt innen maskinlæring som bygger på strukturen og funksjonaliteten til nevrale nettverk. Ordet ”dyp” refererer til bruken av flere *gjemte lag* (*hidden layers*) i nevrale nettverk, noe som gjør dem i stand til å lære komplekse mønstre og representasjoner fra data. Dyplæring er spesielt kraftig for oppgaver som bilde- og talegjenkjenning, samt språkforståelse.

Mens tradisjonelle maskinlæringsmodeller ofte er avhengige av manuell ekstraksjon av funksjoner fra dataene, lærer dyplæringsmodeller disse funksjonene automatisk gjennom lagene i nettverket. Dette gjør dyplæring svært fleksibelt og i stand til å håndtere store mengder ustrukturerte data.

Dyplæring kommer i mange varianter, og vi skal se på noen av de mest anvendte typene:

- *Multiple Layer Perceptron* (MLP): Dette er de ”vanlige” nevrale nettverkene, og ligger i grunn for teori om nevrale nettverk, og dette kalles ofte ”vanilla” eksempelet av nevrale nettverk, altså det enkleste.
- *Convolutional Neural Networks* (CNNs): Effektive for bilder og visuelle data.
- *Recurrent Neural Networks* (RNNs): Brukes for sekvensielle data som tidssrekker og språk. Vi kommer til å se mest på en type av RNN, kalt **Long Short Term Memory modeller**.
- Språkmodeller: Modeller spesialisert på tekstforståelse og generering, slik som transformer-arkitekturen som brukes i moderne språkmodeller.

5.1 Nevrale nettverk

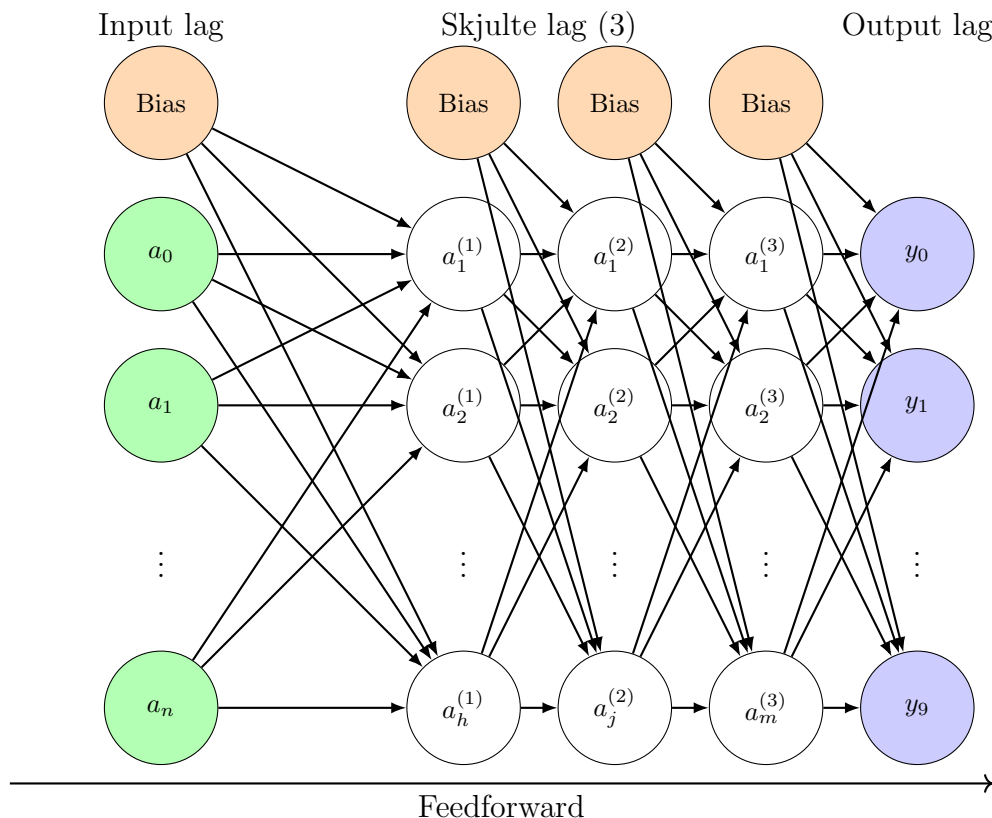
Nevrale nettverk er inspirert av hvordan nevroner fungerer i den menneskelige hjernen. Et nevralt nettverk består av flere lag med ”nevroner” eller enheter som hver utfører enkle beregninger og sender resultatene videre til neste lag.

5.1.1 Struktur av et nevralt nettverk

Et nevralt nettverk består vanligvis av:

- Et *input layer*, som mottar dataene.
- Et eller flere *hidden layers*, hvor de faktiske beregningene og transformasjonene skjer.
- Et *output layer*, som gir det endelige resultatet av beregningene.

I hvert lag vil nevronene motta inngang fra nevronene i forrige lag, vekte denne inngangen, og sende den videre etter en aktiveringsfunksjon som for eksempel *ReLU* eller *sigmoid*.



Figur 13: Nevralt nettverk med tre skjulte lag, inkludert inngangs- og utgangslag

5.2 Nevrale nettverk

Nevrale nettverk kan sees p  som *kjernen* i maskinl ring. I foreg ende kapitler har vi diskutert teknikker fra de mer *tradisjonelle* l ringsteknikkene, som logistisk regresjon, beslutningstr r (og ensemble metoder av disse). La oss n  g  inn i det spennende tema *nevrale nettverk*, hvor vi f r bruk for en del av matematikken fra kapittel 2 og 3!

5.2.1 Arkitektur og matematikken

Nevrale nettverk er bygd opp av lag, kjent som et *Multi-layer Perceptron* (MLP) n r det består av flere lag. Hvert lag består av flere *nevroner*, som er matematiske enheter som transformerer dataen den mottar. Disse enhetene beregner en vektet sum av sine innsignaler og sender videre et utg ende signal, basert p  en aktiveringsfunksjon. La oss se n rmere p  strukturen til nevrale nettverk, som kan deles opp i tre hovedtyper lag:

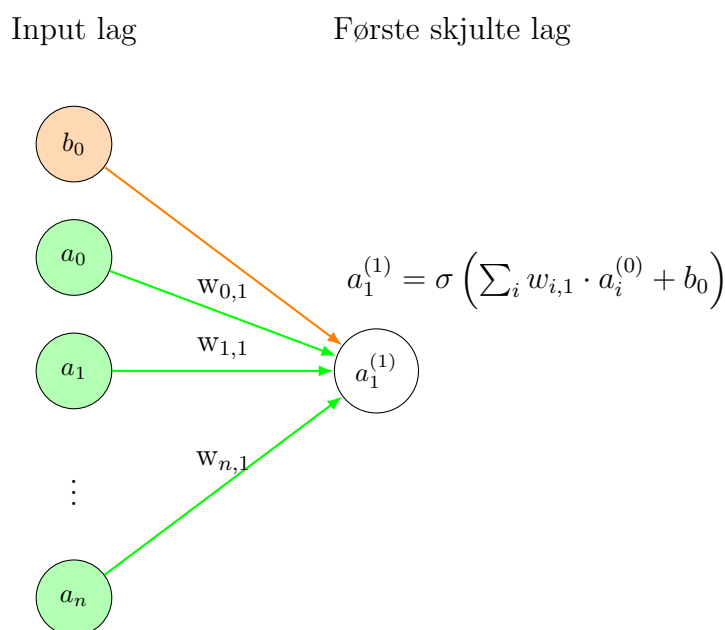
1. **Inngangslag (Input Layer):** Dette f rste laget mottar dataen direkte. Hvert nevron i inngangslaget representerer  n variabel fra datasettet, og dermed tilsvare antallet nevroner i inngangslaget antallet variabler i datasettet. Inngangslaget utf rer ingen beregninger, det er kun et mottakende lag for dataen.
2. **Skjulte lag (Hidden Layers):** Disse lagene er kjernen i nettverket, hvor den faktiske l ringen foreg r. Skjulte lag består av nevroner som utf rer beregninger basert p  dataen fra inngangslaget eller forrige skjulte lag. Hvert nevron i et skjult lag

mottar input fra alle nevronene i det forrige laget, og resultatet beregnes som en vektet sum av disse innkommende signalene, hvor en konstant bias legges til. Summen sendes s  gjennom en *aktiveringsfunksjon*, som for eksempel kan v re sigmoid, ReLU eller tanh. Aktiveringsfunksjonen legger til ikke-linearitet i modellen, noe som gir nettverket evnen til   l re komplekse m nstre i dataen.

En enkel matematisk representasjon for et nevron i skjult lag l kan uttrykkes som:

$$a_j^{(l)} = \sigma \left(\sum_i w_{ij}^{(l)} a_i^{(l-1)} + b_j^{(l)} \right)$$

hvor $a_j^{(l)}$ er utgangen (eller aktiveringen) til nevron j i lag l , $w_{ij}^{(l)}$ er vekten for forbindelsen fra nevron i i forrige lag $l - 1$ til nevron j i lag l , $b_j^{(l)}$ er biasen, og σ er aktiveringsfunksjonen.



Figur 14: Nevralt nettverk med inngangs- og det f rste skjulte laget (kun  n nevron med vekter), hvor σ er en aktiveringsfunksjon.

3. **Utgangslag (Output Layer):** Dette siste laget gir ut nettverkets prediksj n. Antallet nevroner i utgangslaget avh nger av oppgavetypen. For en bin r klassifisering vil utgangslaget typisk ha ett nevron som representerer sannsynligheten for  n klasse, mens et utgangslag med flere nevroner kan representere forskjellige kategorier i et klassifiseringsproblem med flere klasser.

Vekter, biaser og alt det der

De to viktigste komponentene som styrer l ringen i et nevralt nettverk er *vekker* og *biaser*. La oss g  litt mer detaljert inn p  hvordan disse fungerer og hvordan de justeres under treningen.

1. **Vekter (Weights):** Vektene representerer styrken p  forbindelsene mellom nevroner i p f lgende lag. I Figur 13, er det en *vekt* mellom hvert eneste nevron fra et lag, til det neste. Figur 14 viser dette, fra input lag til  n nevron i det f rste skjulte laget. En h y vekt betyr sterkere forbindelse mellom to nevroner, mens en lav vekt gir svakere forbindelse. Matematisk sett er vektene parametere i en matrise, og de justeres kontinuerlig gjennom l ringsprosessen for   minimere nettverkets kostfunksjon (eller tapsfunksjon). Hver vekt representerer et aspekt av nettverkets forst else av dataen, og de finjusteres etter hvert som nettverket eksponeres for flere eksempler.
2. **Biaser (Biases):** Bias er en konstant som legges til f r inputen g r gjennom aktiviseringsfunksjonen. Dette gir nettverket mulighet til   justere terskelen for n r en node skal bli "aktivert," slik at det kan tilpasse seg mer komplekse m nstre i dataene. Bias g r at en node i det nevrale nettverket kan forskyve terskelen for aktivering, noe som gir st rre fleksibilitet. Uten bias ville hver node bare kunne "tenne" n r inputverdien passer n yaktig til vektens kombinasjon, noe som kan v re en begrensning hvis dataene ikke f lger denne eksakte sammenhengen. Bias g r det mulig for nettverket   justere responsen til ulike typer input, selv n r de ligger litt utenfor denne grunnlinjen. Det hjelper nettverket   l re m nstre som ikke n dvendigvis starter fra null, men som trenger et lite "dytt" opp eller ned for   fanges opp riktig.

"Backpropagation" og oppdatering av vekter og biaser

N r nettverket mottar input og produserer en output, sammenlignes denne outputten med den faktiske verdien for   beregne en *feil*, eller en kost. Denne feilen/kosten brukes deretter til   justere vektene og biasene gjennom en prosess som kalles *bakoverpropagasjon* (eng: *backpropagation*).

Bakoverpropagasjon bruker *kjerneregelen* fra derivasjon for   beregne gradientene til kostfunksjonen med hensyn til alle vekter og biaser. Gradientene representerer hvor mye vektene og biasene b r endres for   redusere feilen. Oppdateringen av en vekt skjer som f lger:

$$w_{ij}^{(l+1)} = w_{ij}^{(l)} - \eta \cdot \frac{\partial C}{\partial w_{ij}^{(l)}}$$

der C er kostfunksjonen, og η er l ringsraten. L ringsraten kontrollerer st rrelsen p  oppdateringen per iterasjon. Det er viktig   velge en passende l ringsrate: en for h y rate kan f re til at vi "hopper over" optimal verdi, mens en for lav rate kan g re treningen veldig langsom. Dette har vi diskutert tidligere i kapittel 3.7, se Figur 6.

5.2.2 Feed-forward, "fully-connected" nevrale nettverk

Et **feedforward-nettverk** er en type nevral nettverk der informasjonen str mmer i  n retning, fra inngangslaget til utgangslaget, uten noen tilbakemelding (feedback) fra utgangene tilbake til inngangene. Dette betyr at nevronene i et lag kun p virker nevronene i neste lag, og det er ingen sykluser i nettverket. Feedforward-nettverk er enkle   implementere og brukes ofte i mange applikasjoner, som klassifisering, regresjon, og m nstergjenkjenning.

Under treningen av et feedforward-nettverk brukes metoder som bakoverpropagering (backpropagation) for   justere vektene og biasene i nettverket. M let er   minimere

feilene, eller minimere *kost-funksjonen*, mellom de predikerte outputene og de faktiske dataene i treningsdataene.

Et **tett lag** (ogs  kjent som et "fullt koblet lag") i et nevralt nettverk er en type lag der hver nevron i laget er koblet til hver nevron i det foreg ende laget. Dette gj r at informasjon kan str mme fritt mellom lagene, noe som gir nettverket muligheten til   l re komplekse m nstre og sammenhenger i dataene. Figur 13 viser et nettverk som best r av tette lag (*Dense layers*).

Hver forbindelse mellom nevronene har en tilknyttet **vekt**, som justeres under treningen av nettverket for   forbedre prediksjonene. Hver nevron har ogs  en **bias**, som er en konstant verdi som legges til f r aktiveringsfunksjonen anvendes. Biasen gj r det mulig for nevronene   tilpasse seg bedre til dataene, noe som forbedrer modellens fleksibilitet.

5.2.3 Verkt y og biblioteker

I utviklingen av nevrale nettverk er det flere programmeringsspr k og biblioteker som har blitt standardverkt y for forskere og praktikere. De mest populære inkluderer:

- Tensorflow: Utviklet av Google, TensorFlow er et omfattende bibliotek for numeriske beregninger og dyp l ring. Det gir h y fleksibilitet og kontroll over modellen, samt muligheter for distribuerte beregninger. TensorFlow inkluderer ogs  Keras, et h yere niv  API som forenkler prosessen med   bygge og trene nevrale nettverk.
- Keras: Et brukervennlig og modul rt API som kan brukes med TensorFlow som backend. Det gj r det enkelt   bygge, trene og evaluere nevrale nettverk med lite kode.
- PyTorch er et annet popul rt bibliotek, utviklet av Meta AI. Det er kjent for sin dynamiske beregningsgraf, som gir mer fleksibilitet ved utvikling av nevrale nettverk, spesielt n r det gjelder forskningsform l.
- Scikit-learn: Selv om Scikit-learn ikke er spesifikt designet for nevrale nettverk, tilbyr det enkle modeller og verkt y for forbehandling av data, som kan v re nyttige f r du trener et nevralt nettverk. Samtidig har det enkle pakker for   splitte data inn i trening, validering og test-data, enkelt   bruke logistisk regresjon, beslutningstr r og s  videre.

Dette er for   nevne noen, i kurset kommer vi i hovedsak til   fokusere p  bruken av Tensorflow og Keras.

5.2.4 Bruksomr der i  konomi

Nevrale nettverk har et bredt spekter av anvendelser innen  konomi og finans. Noen f  eksempler p  bruksomr dene er:

- Risikovurdering: I forsikrings- og finanssektoren brukes nevrale nettverk til   vurdere risiko for investeringer, ved   analysere historiske data, markedsf rhold og makro konomiske faktorer.

- Svindeldeteksjon: Nevrale nettverk kan brukes for å finne komplekse sammenhenger mellom transaksjoner for å detektere svindel og hvitvasking.
- Bildeklassifisering: Hvis man skal analysere et havneområde (automatisk), kan man bruke nevrale nettverk for bildedeteksjon - for å se om innkommende båter er militære eller ikke-militære.

5.3 Whisper-modellen

Whisper-modellen, utviklet av OpenAI, er en automatisk talegjenkjenningsmodell som har vist seg å være svært effektiv i å transkribere og oversette tale fra lydfiler [Radford et al., 2022]. Den er trent på et omfattende datasett (680.000 timer med audio og transkripsjon) som inkluderer et bredt spekter av språk og dialekter, noe som gjør den i stand til å håndtere ulike talestiler og aksenter. Den er overraskende god på norsk, og alle dialekter det medfører.

Arkitekturen til Whisper er basert på transformermodeller, som er kjent for sin evne til å behandle sekvensielle data på en svært effektiv måte. Denne modellen kan transkribere tale i sanntid, noe som gjør den ideell for applikasjoner som videokonferanser, live-oversettelser, intervjuer av kvalitative data og så videre. Den kan også brukes til tale-til-tekst-funksjoner, som lar brukere diktere tekst i stedet for å skrive den.

En av de fremste fordelene med Whisper-modellen er dens robusthet mot bakgrunnsstøy og forstyrrelser, noe som gjør den egnet for bruk i varierte lydmiljøer. Det kommer av at treningsdataene ikke bare er perfekt podkast-audio, men lydfiler skrapet fra internett med bakgrunnsstøy.

I tillegg til talegjenkjenning, kan den også oversette mellom flere språk, noe som gir verdifull støtte for kommunikasjon på tvers av språkgrenser. Whisper-modellen representerer et betydelig fremskritt innen automatisk talegjenkjenning, og i dette kurset skal vi få litt *hands on* erfaring med Whisper modellen!

5.3.1 Transformer-modell

Vi kommer ikke til å dypdykke i Whisper modellens arkitektur, men det vi vil dekke er at den består av en transformer-modell.

Det er et kraftig form av nevrale nettverk, og det som gjør Whisper modellen unik - er dens *multi-task* format. Det betyr at modellen kan takle flere oppgaver samtidig. Whisper modellen kan altså sjonglere flere ting av gangen, den kan transkribere tale, mens den finner ut hvilket språk det er, markere når enkelt-ord blir sagt og finne ut om det er faktisk tale eller bakgrunnsstøy, samtidig!

I artikkelen til OpenAI, *Robust Speech Recognition via Large-Scale Weak Supervision* [Radford et al., 2022] fant forskerne ut at Whisper-modellen transkriberer så godt som menneskelig nivå! I tillegg, så utkonkurrerte modellen mennesker på noen ting. Likevel, har Whisper noen baksider. Whisper er trent på korte segmenter med tale, og kan slite på times lange foredrag. Men i AI-verden skjer endringer fort, og Whisper har allerede "jobbet rundt" dette problemet, ved at data behandles som biter (eng: *chunks*) av gangen [Murray, 2024].

5.3.2 Personvern

Et problem med modeller på internett som gjør transkribering, er at man må laste opp datamaterialet sitt. Ofte, for eksempel med intervjudata for en masteroppgave, kan data være konfidensielt og GDPR-regler gjør det vanskelig å bruke modeller på nett hvor du laster opp dataene dine.

I dette kurset, vil vi lære hvordan vi kan bruke verktøy som Google Colab, til at filene våre behandles lokalt og aldri forlater Google Colab-området vårt. Bruker man en arbeidsflyt som dette, hvor dataene i tillegg slettes hver gang Colab-området lukkes, kan man bruke transkriberingsverktøy med noen få linjer kode i Python, samtidig som dataene ikke havner utenfor din kontroll.

5.3.3 Fra lyd til innsikt: Sentimentanalyse

I økonomiske analyser er sjelden transkripsjonen målet i seg selv; den er et middel for å kvantifisere informasjon. I forelesning skal vi se på hvordan vi kan kombinere Whisper med teknikker for naturlig språkbehandling (NLP), spesielt *sentimentanalyse*.

Etter at Whisper har konvertert lyd (f.eks. en tale fra sentralbanksjefen eller kongens tale) til tekst, kan vi bruke biblioteker som `TextBlob` for å måle:

- **Polaritet:** Hvor positiv eller negativ er teksten? (Skala fra -1 til +1).
- **Subjektivitet:** Hvor mye av teksten er basert på fakta kontra meninger? (Skala fra 0 til 1).

Dette gjør det mulig for økonomer å konvertere ”myke” data (tale) til harde tidsserier som kan inngå i økonometriske modeller. For eksempel kan man undersøke om endringer i ”tonen” til Kong Harald.

5.3.4 Praktisk bruk: Modellstørrelser og Hardware

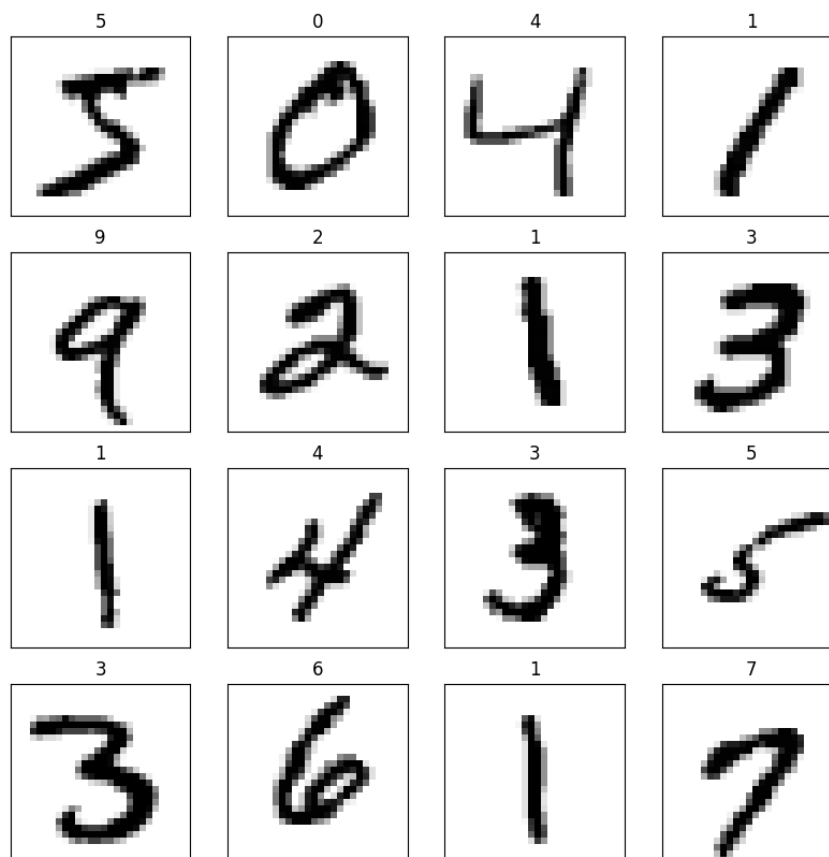
Whisper kommer i ulike størrelser, fra `tiny` til `large`. I dette kurset bruker vi ofte `small` eller `medium` for best mulig balanse mellom kjøretid og nøyaktighet.

Dette diskuteres mer i fysisk forelesning.

5.4 Bildeklassifisering

N  har vi sett p  nevrale nettverk, ved   diskutere Multi-layer Perceptron. N r vi snakket om bilde-data, bruker vi noe som heter **Convolutional Neural Networks**.

MNIST-datasettet er et popul rt datasett innen maskinl ring som består av 70 000 h ndskrevne sifre (0 til 9), hvor hvert bilde har en st rrelse p  28x28 piksler i gr toner, se Figur 15. Dette datasettet har blitt brukt som en standard for testing av enkle klassifikasjonsmodeller.



Figur 15: Eksempler av h ndskrevne tall i MNIST datasettet.

Som vi allerede har sett i de forelesningene, kan en enkel, s kalt "vanilla" multi-layer perceptron (MLP) oppn  ganske gode resultater i   klassifisere disse h ndskrevne tallene. MLP-modeller fungerer godt for enkle billedatasett som MNIST, ettersom de h ndterer bildene som flate vektorer og kan trenes til   gjenkjenne m nstre i pikslene.

M ten vi gjør det p  er at bildene består av piksler, i 28×28 . Hvor hver "celle" består av et tall mellom 0 og 255, fordi bildene er lagret i et 8-bits gr skalaformat (sort/hvitt-bilde). I dette formatet betyr:

- 0 representerer ren svart.
- 255 representerer ren hvit.
- Verdier mellom 0 og 255 representerer ulike nyanser av gr tt, der lavere verdier er m rkere og h yere verdier er lysere.

N r bildene brukes i nevrale nettverk, normaliseres de ofte slik at verdiene ligger mellom 0 og 1. Dette gj res ved   dele hver pikselverdi p  255. Normalisering kan bidra til at nettverket trener mer effektivt, ettersom det holder input-verdiene innenfor et konsistent skalaomr de.

For mer komplekse bildeklassifiseringsoppgaver er det imidlertid n dvendig med modeller som kan utnytte den romlige informasjonen i bildene. Her kommer konvolusjonelle nevrale nettverk (CNN) inn i bildet. I motsetning til MLP bruker CNN spesielle konvolusjonslag som kan fange opp lokale m nstre og teksturer, noe som gj r dem mer egnet til   forst  komplekse bilder der objekter har varierende posisjoner og orienteringer. I de neste kapitlene vil vi se n rmere p  hvordan CNN-er gir en betydelig ytelsesforbedring ved   utnytte romlige relasjoner i bildedata.

5.4.1 Bilder og deres datastruktur

Bilder, hvertfall digitale bilder, best r av et stort antall piksler som hver representerer en liten del av bildet. Hver pixel inneholder fargeinformasjon, og for fargebilder er denne informasjonen vanligvis representert i et RGB-format. RGB st r for R d, Gr nn og Bl , som er de tre prim rfargene i dette fargesystemet. Hver av disse fargene har en intensitet som vanligvis varierer fra 0 til 255, noe som gir over 16 millioner mulige farger.

Datastrukturen til et digitalt bilde kan forst s gjennom konseptet med tensorer. I denne sammenhengen er et bilde representert som en 3D-tensor, hvor dimensjonene er som f lger:

- **H yde (H):** Antall piksler i vertikal retning.
- **Bredde (W):** Antall piksler i horisontal retning.
- **Kanaler (C):** Antall fargekanaler (for et RGB-bilde er dette 3).

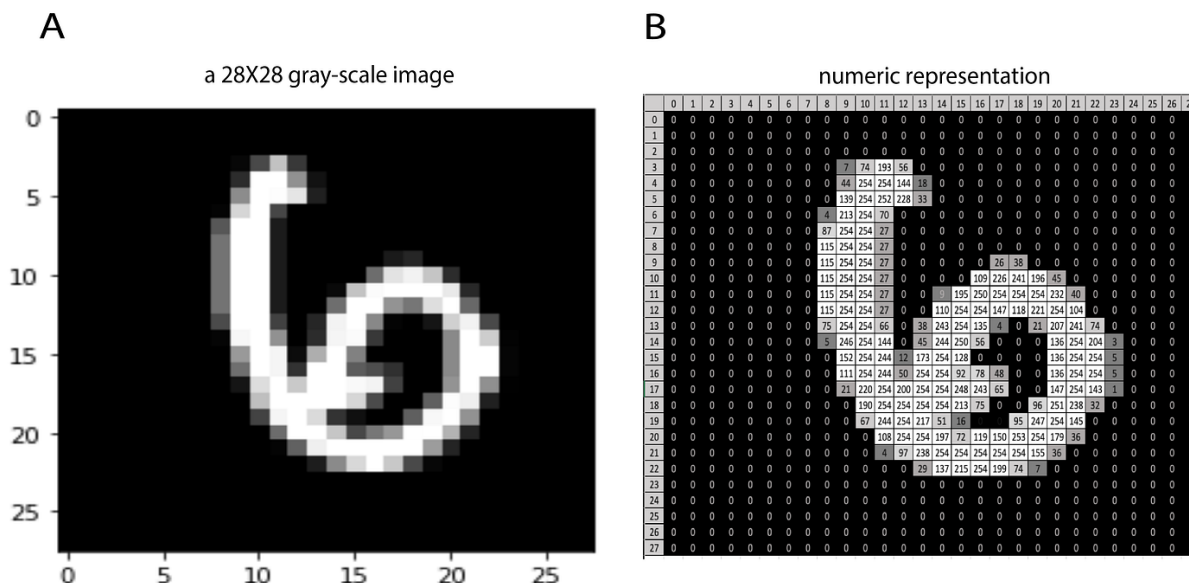
Derfor kan et digitalt bilde med h yde H , bredde W og 3 fargekanaler representeres som en tensor med dimensjonene $H \times W \times C$.

5.4.2 Convolutional Neural Networks

Konvolusjonelle nevrale nettverk, ofte forkortet CNN, er en nettverksarkitektur for dyp l ring, og brukes n r vi diskuterer bilde-data. Et CNN best r av flere lag, derav *dypl ring*, som prosesserer og transformerer et input (som et bilde), og kommer ut i andre enden med et output. Disse CNN'ene kan brukes til bildeprosessering, deteksjon av objekter, men kan ogs  brukes til prediksjon av tekst og lyd.

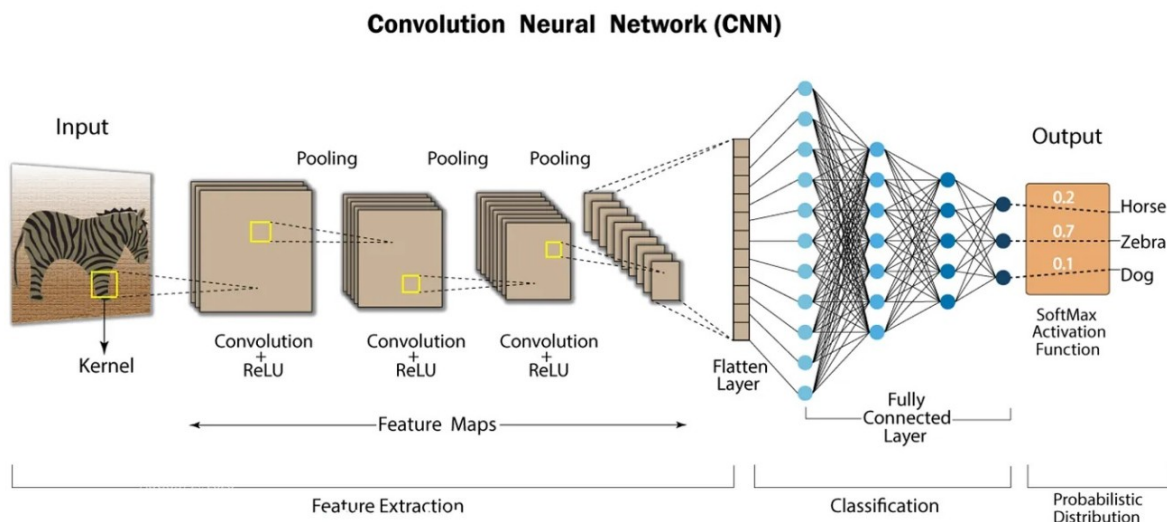
Vi har diskutert fargebilder, som er en rank-3 tensor (3 matriser "opp  hverandre"). Hvis vi ser for oss et gr -skala bilde (rank-2 tensor, med  n matrise), se Figur 16. Dette bildet best r av 28x28 piksler, hvor hver piksel er representert med 8 bits. Hvor da pikselene kan innta verdier fra 0(sort) til 255 (hvitt).

CNN f r navnet sitt fra den matematiske operasjonen, *konvolusjon*. I CNN, implementeres dette gjennom hva vi kaller p  engelsk *feature detector*, *filter* eller en *kernel* - som alle representerer samme konsept. Figur 17 viser et bilde av en zebra, og en kernel. Denne kernel'en, kan sees p  som en matrise (som er mindre enn input bildet), som "beveger" seg over bilde - fra  verst til venstre, og jobber seg mot h yre over bildet og deretter g r den



Figur 16: Illustrasjon av gr skala-bilde [Zammataro, 2022].

ned et hakk. Dette gj res over hele bilde, som gj r at vi f r en ny representasjon av input-bildet, et *feature map* av bildet. Hvordan vi spesifiserer denne kernelen, vil bestemme om den kan oppfatte ulike strukturer i bilde, som gj r at datamaskinen kan gjenkjenne hva bilde er for noe. F.eks. kan kernel gj r at vi gjenkjenner ulike kanter, sirkler i tall - som finner hvilke karakteristikk som er typisk for f.eks. tallet 6. Hvor hvert konvulusjonslag, s  implementeres ofte en aktiveringsfunksjon (for   f  med ikke-line re sammenhenger), f.eks. ReLu funksjonen.



Figur 17: Illustrasjon av CNN.

Etter et konvulusjonslag, m  man implementere et *pooling*-lag. Det brukes til   *down-sample*, alts  nedskalere, feature-map'et vi fikk av konvulusjonslaget. M let her, er at pooling gj r at vi kvitter oss med det som ikke er "viktig" i bilde, og beholder det som er viktig. Hovedgrunnen til   gj re dette er   hindre overtilpasning, og gj re beregningene i

senere lag raskere da bildet er mindre etter pooling-laget. F.eks. bruker vi noe som heter *max pooling*, vil v rt gr skala-bilde g  fra 28x28 piksler til 14x14.

Prosesen som er beskrevet over, gj res igjen og igjen i CNN, for   "f  meg seg" mer abstraksjon i nettverket. Til slutt, s  vil CNN best  av et *flatten layer*, som "flater ut" output'et fra pooling-lagene, se Figur 17. Deretter er det *dense (tette)* lag, som i MLP nettverk vi diskuterte tidligere som i Figur 13. Hvor man til slutt ender opp med et output-lag, som har som m l   klassifisere bilde som nettverket mates med. Det kan f.eks. v re   klassifisere tall mellom 0 og 9 (MNIST datasettet), om det er en hest/zebra/hund, eller om bildet datamaskinen v r tildelt er et milit ert eller sivilt skip.

N  har vi gitt en (meget) kort introduksjon til CNN, hvor vi ikke har diskutert all matematikk bak konvolusjonslag, alternativ man kan gj re ved bruk av kernel, bakpropagering og mer.

5.5 Tidsseriemodeller

I matematikk refererer en tidsserie til en rekke datapunkter som er sortert etter tid. Vanligvis er en tidsserie en sekvens av data som er samlet inn ved jevne mellomrom, og er ofte brukt i økonometri og finans. Lukkeprisen for aksje-data, vil være et typisk eksempel på en tidsserie.

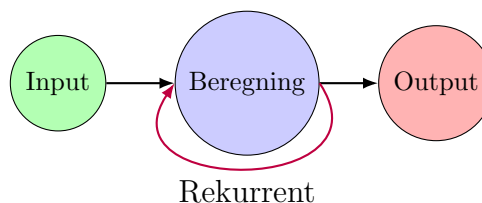
5.5.1 Recurrent neural networks

Recurrent neural network (norsk: *Rekursive nevrale nettverk*), forkortet RNN, er en type nevrale nettverk som er spesielt tilpasset for å arbeide med sekvensiell data. I motsetning til mer tradisjonelle nevrale nettverk (f.eks. MLP), som behandler data i ett enkelt steg, tar RNN-er hensyn til informasjon over flere tidssteg. Dette gjør dem svært nyttige for å analysere og håndtere data som tekst, tale og tidsserier, der rekkefølgen på informasjonen er viktig.

Visste du at?

Ordet *rekurrent* betyr **tilbakevendende**. I sammenheng med rekurrente nevrale nettverk refererer det til nettverkets evne til å ta hensyn til tidligere informasjon i dataserien ved å sende informasjon tilbake til seg selv, noe som muliggjør en sekvensiell avhengighet over flere tidssteg.

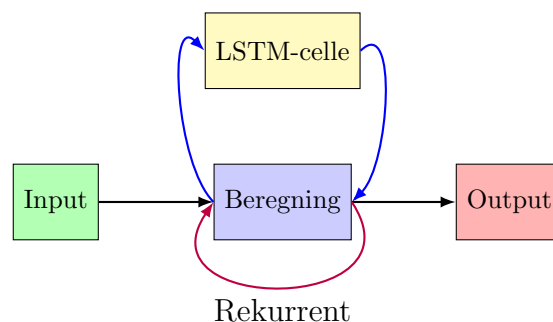
Hvis vi ser på Figur 18 vil RNN lagre output fra første gang den går igjennom bergnings nevronet, og andre gang - etter den tilbakevendende delen. Dette gjør at at nettverket kan "huske" tidligere steg i sekvensen.



Figur 18: Illustrasjon av et rekurrent nevral nettverk med nevroner for input, beregning, og output. Rekurrent, eller tilbakevendende tilbakemelding går fra output tilbake til beregning.

5.5.2 LSTM-modeller

Long short-term memory (LSTM) er en maskinlæringsteknikk som benytter seg av RNN. Som vi diskuterte i delkapittelet over, så kan RNN "huske" tidligere steg - men dette medfører et problem. RNN kan lide av noe (som på engelsk) kalles *long term dependency problem*, som handler om at når nettverket lagrer mer-og-mer informasjon, kan det slite med å tillegne seg ny informasjon.



Figur 19: Illustrasjon av et rekurrent nevraltt nettverk med en LSTM-celle som påvirker beregningen. Tilbakemelding går fra output til beregning, og fra beregning tilbake til LSTM-cellen.

Figur 19 viser at når et input går inn i beregnings-noden, så skjer det en beregning der (for nå, lar vi dette bare være "en beregning") - som leder til et output. Etter dette, vil beregnings-noden få informasjon fra output-noden (tilbakevendende-delen) fra det forrige steget, input fra det nye steget **og** informasjon fra LSTM-cellen.

Hva er en LSTM-celle?

En LSTM-celle består av tre deler som vi kaller for *gates*. Det er:

- **Input gate:** Denne gaten bestemmer hvilken informasjon fra den nåværende inputen som skal lagres i cellens tilstand. Den evaluerer inngangsdataene og avgjør hvor mye av informasjonen som skal beholdes.
- **"Glemme" gate (forget gate):** Denne gaten har som oppgave å bestemme hvilken informasjon fra cellens tidligere tilstand som skal glemmes eller skrotes. Den gir modellen mulighet til å filtrere ut irrelevant informasjon, noe som er et problem i tradisjonelle RNN.
- **Output gate:** Denne gaten bestemmer hva som skal sendes videre fra LSTM-cellen til det neste laget i nettverket. Den evaluerer cellens interne tilstand og bestemmer hvilke deler av informasjonen som skal brukes til å generere output.

Alle komponentene i en LSTM-celle får en verdi mellom 0 og 1, hvor 0 betyr at *alt glemmes*, mens 0.5 betyr at noe glemmes og noe tas vare på, og 1 betyr at man tar vare på all informasjonen.

5.5.3 Når brukes dette?

LSTM modeller kan brukes til mange ting, som

- Chat-botter, hvor de kan glemme noe informasjon etterhvert, men kanskje må lagre annen informasjon.
- Prediksjon i tidsserie-data, som er det vi kommer til å fokusere mest på i dette kurset når det gjelder LSTM-modeller.
- Tale-gjenkjenning
- og mange, mange fler.

6 Så hvorfor er dette interessant for økonomer

Så langt i kompendiet har vi diskutert mange maskinlæringsteknikker, og deres detaljer. Det er viktig å ha en forståelse for fagfeltet, for å kunne anvende det i praksis.

Maskinlæring har blitt stadig viktigere innen økonomi og finans, og teknikker som brukes i nevrale nettverk kan tilby store fordeler for økonomer som jobber med både teoretiske modeller og prediksjon. Mens økonomer tidligere har hatt en tendens til å ikke benytte seg av maskinlæring på grunn av metodiske forskjeller fra tradisjonell økonometrisk tilnærming, har det skjedd en gradvis aksept for disse metodene [Hull, 2021]. Maskinlæring, med sitt fokus på prediksjon gjennom komplekse, ikke-lineære modeller, tilbyr en kraftig tilnærming som kan **komplementere** (ikke nødvendigvis erstatte) økonometriske metoder.

Hvor dens anvendelse er mange, fra finansielle avgjørelser, deteksjon av svindel/hvitvasking og mange flere samfunnsøkonomiske anvendelser.

7 Hvorfor er maskinlæring interessant for økonomer?

Mens tradisjonell økonometri ofte fokuserer på å isolere årsakssammenhenger (kausalitet) for å teste økonomisk teori, handler maskinlæring i stor grad om å maksimere prediksjonskraft. For en økonom kan dette skiftet i fokus virke fremmed, men det åpner også opp for å løse problemer som tidligere var vanskelige og/eller umulige.

Vi kan oppsummere motivasjonen for å lære maskinlæring i tre hovedkategorier: Bedre prediksjoner, håndtering av ustrukturerte data, og fange opp ikke-lineære sammenhenger.

OBS: *Vi kunne nok ha diskutert mange kategorier her, men tok tre viktige.*

7.1 Prediksjon som et beslutningsverktøy

Mange økonomiske beslutninger avhenger av en *nøyaktig* (eller, *så god som mulig*) prognose av fremtiden, snarere enn en dyp forståelse av de underliggende mekanismene.

- **Finansmarkeder:** For en porteføljeforvalter er det ofte viktigere at en modell treffer godt på morgendagens aksjekurs (høy R^2 out-of-sample, lav MSE og andre metrikker), enn at parameterestimaterne er forventningsrette (unbiased).
- **Det å ta valg:** [Kleinberg et al., 2015] argumenterer for at mange politiske problemer egentlig er prediksjonsproblemer. For eksempel: En dommer som skal avgjøre om en tiltalt skal varetektsfengsles, forsøker egentlig å *predikere* sannsynligheten for at personen begår ny kriminalitet eller rømmer. Maskinlæringsmodeller har vist seg å kunne gjøre slike vurderinger mer presist enn menneskelig skjønn alene.

7.2 Den ustrukturerte datarevolusjonen

Den kanskje største verdien for moderne økonomer ligger i muligheten til å bruke *ustrukturerte data*. Tradisjonell økonometri forutsetter ofte at dataene våre kommer ferdig strukturert i regneark (paneldata eller tidsserier). Verden består imidlertid i stor grad av tekst, bilder og lyd.

- **Tekst som data (NLP):** Sentralbanker (og andre banker) sitter p  enorme mengder tekst (referater, taler, rapporter). Ved hjelp av spr kmodeller – som vi skal se n rmere p  med eksempler som Whisper – kan vi kvantifisere ”sentimentet” i disse tekstene. Dette gjør det mulig   kvantifisere ”hvor positiv”, eller ”hvor negativ” en tekst er. Det gjør at man faktisk kan gj re ”tekst om til tall”, og hente nyttig informasjon i fra dette..
- **Bilder som data:** Vi kan bruke satellittbilder av opplyste omr der om natten (night lights) som en proxy for  konomisk vekst i utviklingsland der BNP-tall er up litelige. Maskinl ring, spesielt Convolutional Neural Networks (CNN), gjør det mulig   trekke ut slik informasjon automatisk.

7.3 Komplexitet og ikke-linearitet

 konomisk teori er ofte bygget p  line re approksimasjoner fordi de er matematisk h ndterbare. Virkeligheten er derimot sjelden line r.

En av de viktigste teoretiske begrunnelsene for   bruke nevrale nettverk i  konomi, er deres evne til   modellere vilk rlige funksjoner uten at vi trenger   spesifisere formen p  forh nd.

Dette er et teorem, som p  egnelsk ofte kalles **Universal Approximation Theorem**.

Teoremet sier at et *feedforward* nevral nettverk med kun ett skjult lag (gitt at det har nok nevroner og en passende ikke-line r aktiveringsfunksjon) kan tiln rme enhver kontinuerlig funksjon p  et avgrenset omr de.

Universal Approximation Theorem

La $C(K)$ v re rommet av kontinuerlige funksjoner p  en kompakt delmengde $K \subseteq \mathbb{R}^n$.

For enhver kontinuerlig funksjon $f \in C(K)$ og for enhver feilmargin $\epsilon > 0$, eksisterer det et nevral nettverk $\hat{f}$ med ett skjult lag slik at:

$$|\hat{f}(x) - f(x)| < \epsilon \quad \text{for alle } x \in K$$

Hva betyr dette i praksis for en  konom?

I tradisjonell  konometri m  vi ofte gjette p  den funksjonelle formen. Vi antar for eksempel:

$$y = \beta_0 + \beta_1 x + \beta_2 x^2 + \epsilon$$

Hvis den ”sanne” sammenhengen i verden egentlig er logaritmisk, sinus-formet eller har br  knekkpunkter, vil denne modellen feile, uansett hvor mye data vi har.

Universal Approximation Theorem forteller oss at nevrale nettverk er (p  godt norsk) *universelle funksjonsapproksimatorer*. S  lenge vi har nok data og nok nevroner (bredde i det skjulte laget), garanterer teoremet at det *finnes* et nettverk som kan etterligne den sanne funksjonen $f(x)$ med s  h y n yaktighet (ϵ) vi  nsker. Vi trenger ikke vite om sammenhengen er line r eller ikke p  forh nd; modellen lærer formen av seg selv gjennom treningsprosessen.

Likevel, er det viktig   huske p  at for at dette skal funke m  man ha data - masse data!

Uansett er maskinlæring spesielt nyttig til f.eks. finansielle tidsserier (som ofte har komplekse avhengigheter) eller i makroøkonomiske prognoser der samspillet mellom variabler endrer seg over tid.

Oppsummert gir maskinlæring økonomer en større verktøykasse. Det erstatter ikke behovet for økonomisk forståelse og økonometriske metoder, men det gir oss muligheten til å analysere mer komplekse data og lage bedre prognoser i en stadig mer digitalisert økonomi.

8 Praktisk bruk i Python

De fysiske forelesningene og seminarene vil være der dere får lære mest om praktisk bruk av maskinlæring i Python. Likevel, vil denne seksjonen vise noen praktiske eksempler med Python kode.

8.1 Datasett

En viktig del av maskinlæring er å håndtere dataen på en effektiv måte. Her ser vi på hvordan vi kan laste inn, utforske og prosessere data for bruk i maskinlæringsmodeller, samt hvordan vi kan bruke `scikit-learn` for å dele opp datasettet i trenings- og testsett.

8.2 Innlasting av data

Den vanligste måten å håndtere data på i Python er ved bruk av `pandas`-biblioteket, som tilbyr en fleksibel struktur kalt `DataFrame`. En `DataFrame` kan inneholde tabulære data, noe som gjør det enkelt å utføre analyser og prosessering. La oss starte med å laste inn et eksempeldata, som vi kan anta ligger i en CSV-fil.

```
1 import pandas as pd
2
3 # Laster inn data fra en CSV-fil
4 df = pd.read_csv("data.csv")
5
6 # Ser på de første radene av dataen
7 print(df.head())
```

I koden over laster vi inn dataen fra en fil kalt `data.csv` og bruker `head()` for å se de første radene. Dette gir en rask oversikt over kolonner og verdier.

8.3 Utforske dataen

Etter å ha lastet inn dataen er det viktig å få en forståelse av datasettet. Dette kan innebære å se på kolonnenavn, datatype, og en statistisk oppsummering av numeriske variabler.

```
1 # Informasjon om datasettets kolonner og datatyper
2 print(df.info())
3
4 # Statistisk oppsummering av numeriske kolonner
5 print(df.describe())
```

Disse kommandoene hjelper oss med å avdekke eventuelle manglende verdier og om det finnes variabler som må skaleres eller prosesseres ytterligere.

8.4 Dataprosessering

I maskinlæring er kvaliteten på dataen ofte avgjørende for modellens ytelse. Dette diskuterte vi tidligere i kompendiet som *shit in, shit out*. Her er noen vanlige steg for å prosessere dataen:

1. **Behandling av manglende verdier:** Ofte mangler data i noen rader eller kolonner, og vi kan enten fylle disse verdiene eller fjerne radene.

```
1 # Fyller manglende verdier med median
2 df.fillna(df.median(), inplace=True)
3
4 # Alternativt kan vi droppe rader med manglende verdier
5 df.dropna(inplace=True)
```

2. **Skalering av data:** Mange maskinlæringsalgoritmer fungerer bedre når data er skalert til et lignende område, for eksempel mellom 0 og 1.

```
1 from sklearn.preprocessing import StandardScaler
2
3 # Skalerer en dataframe
4 scaler = StandardScaler()
5 df_scaled = scaler.fit_transform(df)
```

3. **Encoding av kategoriske variabler:** Hvis datasettet inneholder kategoriske variabler (for eksempel tekstverdier som "Ja" eller "Nei" eller andre kategoriske verdier som "Rød", "Grønn", "Blå"), må disse ofte gjøres om til numeriske verdier for å kunne brukes i maskinlæringsmodeller. Dette kan gjøres på flere måter:

- **One-hot encoding:** Denne metoden lager nye kolonner, én for hver kategori, og fyller disse kolonnene med 1 eller 0 avhengig av om observasjonen tilhører den spesifikke kategorien. Dette er spesielt nyttig når det ikke finnes en naturlig rekkefølge mellom kategoriene. `pandas` tilbyr en enkel måte å gjøre dette på med funksjonen `get_dummies`.

```

1 import pandas as pd
2 # One-hot encoding for en kolonne kalt 'kategorie'
3 df = pd.get_dummies(df, columns=['kategorie'])

```

For eksempel, hvis kolonnen `kategori_kolonne` har verdiene R d, Gr nn, og Bl , vil dette lage tre nye kolonner: `kategori_R d`, `kategori_Gr nn`, og `kategori_Bl `, der hver rad vil inneholde 1 i kolonnen for sin kategori og 0 i de andre.

kategori	Bl�	R�d	Gr�nn	Bl�	R�d
kategori_Bl�	1	0	0	0	0
kategori_R�d	0	1	0	1	0
kategori_Gr�nn	0	0	1	0	0

Figur 20: Illustrerer hva One-Hot Encoding gjør med 'kategori' kolonnen

- **Label encoding av input:** Noen ganger er det ikke  nskelig   bruke *one-hot encoding*, f.eks. over diskuterte vi det ved fargene bl , gr nn og r d. Ved one-hot encoding s  opprettes det tre kolonner, som har verdien 0 eller 1, basert p  kolonne-navn og om den raden representerer fargen eller ikke. Men hvis vi har data hvor en naturlig ordning eller rangering, kan vi bruke **ordinal encoding**. I kode-eksempelet under bruker vi `sklearn.preprocessing` og `OrdinalEncoder()` for   beholde rekkef lgen i variabelen. "Liten" blir lik 0, "Middels" blir lik 1 og "Stor" blir lik 2.

```
1 from sklearn.preprocessing import OrdinalEncoder
2
3 # Lager et eksempel-datasett
4 data = {'størrelse': ['Liten', 'Stor', 'Middels', 'Liten',
5                     'Stor', 'Middels', 'Liten']}
6 df = pd.DataFrame(data)
7
8 # Bruker OrdinalEncoder med spesifiserte kategorier og ordning.
9 encoder = OrdinalEncoder(categories=[['Liten', 'Middels', 'Stor']])
10
11 # "Fit and transform" av 'størrelse' kolonnen
12 df['størrelse'] = encoder.fit_transform(df[['størrelse']])
```

Dette har mange fordeler i maskinlæring, for eksempel bruker vi beslutningstrær vil denne ordningen gjøre at treet kan forstå forskjellen på variablene på en måte som "gir mening" og vi bevarer relasjonen mellom variablene. Men, det kan føre til at noen modeller "tror" det er en lineær sammenheng mellom variabler som kan medføre implikasjoner.

- **Label encoding av output:** Hvis man har output-variabler som er kategoriske, kan man benytte seg av `LabelEncoder()` fra `sklearn.preprocessing`, som tilegner et heltall til en kategori. F.eks hvis. en kolonne `df['output']` har verdiene "Kina", "India", "USA" og "Brazil". Vil koden under tilegne en verdi til hvert av landene, som "Brazil" = 0, "India" = 1, "Kina" = 2 og "USA" = 3.

```
1 from sklearn.preprocessing import LabelEncoder
2 # Label encoding for en kolonne kalt 'output'
3 encoder = LabelEncoder()
4 df['output'] = encoder.fit_transform(df['output'])
```

Her er det viktig å merke seg at `LabelEncoder` egner seg å "encode" outputvariabler/målvariabel, ikke input. Legg også merke til at den sorterer i alfabetisk rekkefølge.

8.5 Splitte data i trenings- og testsett

En vanlig praksis i maskinlæring er å dele datasettet i et treningssett og et testsett. Treningssettet brukes til å lære opp modellen, mens testsettet brukes til å evaluere ytelsen.

```
1 from sklearn.model_selection import train_test_split
2
3 # Anta at "y" er målvariabelen/output, og resten er input-funksjoner
4 X = df.drop("målvariabel", axis=1)
5 y = df["målvariabel"]
6
7 # Del opp i trenings- og testsett
```

```
8 X_train, X_test, y_train, y_test = train_test_split(X, y,  
9 test_size=0.2, random_state=42)
```

8.6 Line r regresjon

Line r regresjon brukes for   modellere forholdet mellom en avhengig variabel og en eller flere uavhengige variabler. Modellen finner den beste linjen (eng: *best fit*) som passer til dataene. Den generelle formelen for enkel line r regresjon er:

$$y = \beta_0 + \beta_1 x + \epsilon$$

Her er y den avhengige variabelen, x er den uavhengige variabelen, β_0 er interceptet, β_1 er stigningstallet (koeffisienten) og ϵ er feilleddet.

I Python kan vi bruke `scikit-learn` for   implementere line r regresjon. Her er et eksempel p  hvordan man kan bruke line r regresjon til   forutsi verdien av en variabel basert p  en annen:

```
1 import numpy as np  
2 import matplotlib.pyplot as plt  
3 from sklearn.linear_model import LinearRegression  
4  
5 # Eksempeldata: Uavhengig variabel (x) og avhengig variabel (y)  
6 x = np.array([1, 2, 3, 4, 5]).reshape(-1, 1)  
7 y = np.array([2, 4, 5, 4, 5])  
8  
9 # Initialiser og trener v r modell  
10 model = LinearRegression()  
11 model.fit(x, y)  
12  
13 # Prediksj n  
14 y_pred = model.predict(x)  
15  
16 # Visualisering  
17 plt.scatter(x, y, color='blue', label='Data')  
18 plt.plot(x, y_pred, color='red', label='Line r regresjon')  
19 plt.xlabel('X')  
20 plt.ylabel('Y')  
21 plt.legend()  
22 plt.show()  
23  
24 # Utskrift av koeffisientene  
25 print(f"Skj ringspunkt (beta_0): {model.intercept_}")  
26 print(f"Stigningstall (beta_1): {model.coef_}")
```

I koden over har vi definert et enkelt datasett med  n uavhengig variabel x og en avhengig variabel y . Vi benytter `LinearRegression` fra `scikit-learn` for   finne den beste line re modellen. Deretter visualiserer vi resultatene med `matplotlib`, og skriver ut modellens koeffisienter.

8.7 Logistisk regresjon

Logistisk regresjon brukes for klassifikasjon, hvor vi  nsker   predikere en bin r utfall (typisk eksempel er om noe **skjer** eller **ikke**). Den logistiske funksjonen (sigmoid-funksjonen) konverterer output til   bli mellom 0 og 1:

$$P(y = 1|x) = \frac{1}{1 + e^{-(\beta_0 + \beta_1 x)}}$$

Her er $P(y = 1|x)$ sannsynligheten for at hendelsen y skjer gitt x , og β_0, β_1 er modellens parametre.

Kodeforslag:

```
1 import pandas as pd
2 from sklearn.linear_model import LogisticRegression
3 from sklearn.model_selection import train_test_split
4 from sklearn.metrics import accuracy_score
5
6 # Importerer datasettet (som er brukt tidligere i kurset)
7 df = pd.read_csv('Fraud.csv')
8 df.head()
9
10 # Kvitter oss med "un dvendige variabler"
11 df = df.drop(['nameOrig', 'nameDest', 'Unnamed: 0',
12             'Date of transaction', 'type', 'branch',
13             'Acct type', 'Time of day'], axis = 1)
14
15 # Kvitter oss med rader hvor det er NaN-verdier
16 df = df.dropna()
17
18 # Definerer output-variabelen (alts , dens kolonne-navn)
19 target = 'isFraud'
20
21 # X skal best  av alle kolonner unntatt output-variabelen
22 X = df.drop(columns=[target_column])
23 y = df[target_column] # M lvariabelen/output
24
25 # Del datasettet i trening og testsett
26 X_train, X_test, y_train, y_test = train_test_split(X, y,
27                                                     test_size=0.3, random_state=42)
28
29 # Initialiser og tren modellen
30 logreg = LogisticRegression()
31 logreg.fit(X_train, y_train)
32
33 # Prediksj n p  testsettet
34 y_pred = logreg.predict(X_test)
35
36 # Beregn n yaktighet
37 accuracy = accuracy_score(y_test, y_pred)
38
```

```
39 # Skriv ut nøyaktigheten/accuracy
40 print(f"Accuracy er: {accuracy * 100:.2f}%")
```

I dette eksempelet bruker vi logistisk regresjon for å gjennomføre binær klassifikasjon på datasettet `Fraud.csv`. Vi trener altså en logistisk regresjonsmodell på treningsdataene, og evaluerer den på testsettet for å beregne nøyaktigheten.

8.8 Beslutningstrær

I eksempelet under vises vi hvordan vi lager et beslutningstre, for å klassifisere **svindel** eller **ikke-svindel** fra `Fraud.csv` datasettet.

```
1 import pandas as pd
2 from sklearn.tree import DecisionTreeClassifier
3 from sklearn.model_selection import train_test_split
4 from sklearn.metrics import accuracy_score
5
6 # Importerer datasettet
7 df = pd.read_csv('Fraud.csv')
8 df.head()
9
10 # Kvitter oss med "unødvendige variabler"
11 df = df.drop(['nameOrig', 'nameDest', 'Unnamed: 0',
12             'Date of transaction', 'type', 'branch',
13             'Acct type', 'Time of day'], axis = 1)
14
15 # Kvitter oss med rader hvor det er NaN-verdier
16 df = df.dropna()
17
18 # Definerer output-variabelen (altså, dens kolonne-navn)
19 target_column = 'isFraud'
20
21 # X skal bestå av alle kolonner unntatt output-variabelen
22 X = df.drop(columns=[target_column])
23 y = df[target_column] # Målvariabelen/output
24
25 # Del datasettet i trening og testsett
26 X_train, X_test, y_train, y_test = train_test_split(X, y,
27                                                    test_size=0.3, random_state=42)
28
29 # Initialiser og tren modellen med DecisionTreeClassifier
30 dtree = DecisionTreeClassifier(random_state=42)
31 dtree.fit(X_train, y_train)
32
33 # Prediksjon på testsettet
34 y_pred = dtree.predict(X_test)
35
36 # Beregn nøyaktighet
37 accuracy = accuracy_score(y_test, y_pred)
```

```
38  
39 # Skriv ut nøyaktigheten/accuracy  
40 print(f"Accuracy er: {accuracy * 100:.2f}%")
```

Her er det viktig å vite at vi ser kun på *accuracy*, og for å få et mer konkret bilde av hva som foregår i dette beslutningstreet må man visualisere treet. Kanskje må man gjennomføre pruning, som er nevnt tidligere i kompendiet.

Her er ikke nødvendigvis accuracy det beste målet på modellens ytelse, **hvorfor det?**

8.9 Random Forests

[Selvstudium og gjennomgås i kodeøkter]

8.10 Boosting

[Selvstudium og gjennomgås i kodeøkter]

8.11 Nevrale nettverk

[Selvstudium og gjennomgås i kodeøkter]

8.11.1 Epochs

[Selvstudium og gjennomgås i kodeøkter]

8.11.2 Regulariseringsteknikker

[Selvstudium og gjennomgås i kodeøkter]

8.12 Whisper

[Selvstudium og gjennomgås i kodeøkter]

8.13 CNN

[Selvstudium og gjennomgås i kodeøkter]

8.14 LSTM

[Selvstudium og gjennomgås i kodeøkter]

9 Oppgaver

9.1 Oppgaver

1. Beskriv hva som menes med vektorer og matriser i maskinl ring. Gi eksempler p  hvordan de brukes i praktiske anvendelser.
2. Forklar betydningen av normer i line r algebra og hvordan de anvendes i maskinl ring.
3. Hva er transponering av en matrise? Illustrer med et eksempel og forklar hvordan denne operasjonen kan v re nyttig.
4. Beskriv skalar multiplikasjon og gi et eksempel p  hvordan det brukes i sammenheng med maskinl ring.
5. Forklar hva tensorer er, og hvordan de skiller seg fra matriser. Gi eksempler p  anvendelser av tensorer i maskinl ring.
6. Forklar hva som menes med gradienten av en funksjon, og hvorfor den er viktig i optimeringsproblemer i maskinl ring.
7. Beskriv hvordan gradient descent fungerer, og forklar hvordan l ringsraten p virker prosessen.
8. Gradient av flervariabel funksjon.
 - a) Beregn gradienten av funksjonen $f(x, y) = x^2 + y^2 - 4x - 6y + 10$.
 - b) Start fra punktet $(x_0, y_0) = (0, 0)$ og bruk gradient descent med l ringsrate $\eta = 0.1$ for   finne et lokalt minimum av funksjonen. Utf r tre iterasjoner og vis utregningene.
9. Funksjonen $f(x, y) = x^2 + y^4 - 3xy + 7$ skal minimeres.
 - a) Beregn gradienten av $f(x, y)$.
 - b) Bruk gradient descent fra startpunkt $(x_0, y_0) = (1, 1)$ og l ringsrate $\eta = 0.01$ til   finne et lokalt minimum etter fire iterasjoner.
10. Forklar forskjellen mellom L1- og L2-normer.
 - a) Beregn L1- og L2-normene til vektoren $\mathbf{v} = \begin{bmatrix} 3 \\ -4 \\ 1 \end{bmatrix}$.
 - b) Forklar hvordan regularisering ved bruk av disse normene p virker maskinl ringsmodeller.
11. **Matrisemultiplikasjon i et enkelt nevral nettverk:** Gitt en vektormatrise for input- og vektorkombinasjonene for et enkelt nevral nettverk:

- Inputvektor:

$$\mathbf{x} = \begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix}$$

- Vektormatrise:

$$\mathbf{W} = \begin{bmatrix} 0.1 & 0.2 & 0.3 \\ 0.4 & 0.5 & 0.6 \end{bmatrix}$$

- Biasvektor:

$$\mathbf{b} = \begin{bmatrix} 0.1 \\ 0.2 \end{bmatrix}$$

Utfør følgende oppgaver:

- (a) Beregn output fra det lineære laget ved hjelp av:

$$\mathbf{z} = \mathbf{W}\mathbf{x} + \mathbf{b}.$$

- (b) Bruk en sigmoid-aktiveringsfunksjon:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

og beregn outputen for det første laget i nettverket.

9.2 L sningsforslag

1. Vektorer og matriser er sentrale i representasjonen av data. Vektorer kan representere datapunkter, mens matriser kan representere datasett eller transformasjoner. For eksempel brukes matriser i nevrale nettverk for   lagre vektene.
2. Normer er viktige for   m le lengden av vektorer, noe som er nyttig i optimalisering og for   evaluere tap. L2-norm og L1-norm er to vanlige normer brukt i maskinl ring.
3. Transponering bytter plass p  radene og kolonnene i en matrise. For eksempel, hvis vi har matrisen

$$A = \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}$$

s  er den transponerte matrisen A^T :

$$A^T = \begin{pmatrix} 1 & 3 \\ 2 & 4 \end{pmatrix}$$

Transponering kan v re nyttig i beregninger som involverer dot-produktet mellom vektorer.

4. Skalar multiplikasjon er n r en matrise eller vektor multipliseres med et tall. For eksempel, hvis vi har $k = 2$ og vektoren

$$x = \begin{pmatrix} 1 \\ 2 \end{pmatrix}$$

s  er $k \cdot x = \begin{pmatrix} 2 \\ 4 \end{pmatrix}$. Dette brukes for   justere vektorer i modeller.

5. Tensorer er en utvidelse av matriser og vektorer til h yere dimensjoner. For eksempel, et gr skalabilde representeres som en rank-2 tensor (2D-matrise), mens et fargebilde (RGB) representeres som en rank-3 tensor (h yde $\times$ bredde $\times$ kanaler).

Videre er det ikke lagd l sningsforslag, men oppfordrer dere til   l se oppgavene etter beste evne selv. I tillegg, er oppgavene her en god trening p    forst  og forklare konsepter som kan bli aktuelle p  muntlig eksamen.

References

- [Aase, 2022] Aase, M. J. (2022). Predicting persistent weak layers in maritime regions in norway using meteorological parameters. Master’s thesis, Norwegian University of Science and Technology, Trondheim, Norway. Supervisor: Jo Eidsvik.
- [Curry, 2021] Curry, R. (2021). The complete guide to random forests: Part 2. Accessed: 2024-10-31.
- [Deng et al., 2021] Deng, H., Zhou, Y., Wang, L., and Zhang, C. (2021). Ensemble learning for the early prediction of neonatal jaundice with genetic features. *BMC Medical Informatics and Decision Making*, 21.
- [Goodfellow et al., 2016] Goodfellow, I., Bengio, Y., and Courville, A. (2016). *Deep Learning*. MIT Press, Cambridge, MA.
- [Hastie et al., 2009] Hastie, T., Tibshirani, R., and Friedman, J. (2009). *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*. Springer Series in Statistics. Springer, New York, 2nd edition. Access provided by UiT The Arctic University of Norway.
- [Hull, 2021] Hull, I. (2021). *Machine Learning for Economics and Finance in TensorFlow 2: Deep Learning Models for Research and Industry*. Apress, Nacka, Sweden.
- [James et al., 2023] James, G., Witten, D., Hastie, T., Tibshirani, R., and Taylor, J. (2023). *An Introduction to Statistical Learning with Applications in Python*. Springer Nature Switzerland AG.
- [Kleinberg et al., 2015] Kleinberg, J., Ludwig, J., Mullainathan, S., and Obermeyer, Z. (2015). Prediction policy problems. *American Economic Review*, 105(5):491–495.
- [Lang, 2022] Lang, N. (2022). From vectors to tensors: Exploring the mathematics of tensor algebra. Accessed: 2024-11-01.
- [Mishra, 2023] Mishra, M. (2023). Stochastic gradient descent: A basic explanation. *Medium*. Accessed: 2024-10-17.
- [Murray, 2024] Murray, C. (2024). Whisper audio to text: Transcribing long audio files with python. Accessed: 2024-10-30.
- [Ogbemi, 2021] Ogbemi, M.-E. (2021). What is overfitting in machine learning? *freeCodeCamp*. Accessed: 2024-10-17.
- [Radford et al., 2022] Radford, A., Kim, J. W., Xu, T., Brockman, G., McLeavey, C., and Sutskever, I. (2022). Robust speech recognition via large-scale weak supervision. *OpenAI*. <https://cdn.openai.com/papers/whisper.pdf>.
- [Zammataro, 2022] Zammataro, L. (2022). One-vs-all logistic regression for image recognition: How to implement a machine learning python code for classifying images. *Towards Data Science*. <https://towardsdatascience.com>.